

1. Context

1.1. Introducció

El projecte “*Optimització del backend a una aplicació cloud*” és un Treball de Final de Grau (TFG) d’Enginyeria Informàtica, especialitzat en la menció d’Enginyeria del Software. Realitzat a la Facultat d’Informàtica de Barcelona (FIB), segueix l’estructura de treball de la modalitat d’empresa. Busca aprofundir en un problema real i amb aplicacions pràctiques immediates per tal de demostrar i aplicar els aprenentatges obtinguts al llarg del grau universitari.

L’empresa en la qual el treball ha estat dut a terme és l’Institut de Tecnologia de la Construcció de Catalunya (ITeC) [\[1\]](#). Es tracta d’una fundació privada sense ànim de lucre, creada el 1978 i centrada en el sector de la construcció. L’objectiu de l’ITeC és fomentar el creixement i la competitivitat dels agents locals del sector a través de la generació i transferència d’informació, a través del coneixement i sobretot a través de la prestació de serveis tecnològics.

Per aquest motiu disposen d’un gran equip d’enginyers, tècnics i informàtics treballant per garantir el bon desenvolupament i manteniment d’aquests serveis. Des de l’equip de Desenvolupament d’Aplicacions Informàtiques es gestionen diferents aplicacions i és des d’on aquest projecte ha estat portat a terme i on els diferents professionals m’han pogut assessorar i ajudar durant el transcurs del treball.

1.2. Identificació del problema

Planificar, certificar el seguiment o gestionar el manteniment d’obres de grans edificis, hospitals o carreteres resulta una tasca complexa. La gran quantitat d’informació i el gran detall de cadascun dels processos involucrats condiciona molt els càlculs i l’impacte d’un petit canvi pot afectar significativament el procés. Hi ha molts factors que cal tenir en compte: la normativa vigent, el registre d’operacions realitzades, la mà d’obra contractada, etc. Per aquest motiu és vital tenir eines preparades i adaptes a les necessitats dels professionals encarregats d’aquestes construccions.

Tenint-ho en compte, des de fa més de vint anys, ITeC ofereix l’aplicació anomenada Temps Cost Qualitat (TCQ) és un *software* utilitzat per gestionar de forma col·laborativa i integral tot el cicle de vida d’un projecte constructiu. Es tracta d’una aplicació d’escriptori, organitzada amb diferents mòduls que faciliten a través del *software* aquelles tasques i gestions essencials.

Recentment, seguint les tendències actuals, l'empresa ha anat més enllà i s'està centrant en una nova versió anomenada TCQi. Es tracta d'una adaptació de l'anterior però pensada per treballar en el núvol. El gruix de feina de l'equip recau en el trasllat d'aquests mòduls de TCQ cap a TCQi, ja que idealment la versió d'escriptori deixarà de rebre manteniment en els anys vinents. De moment, fins que no s'acabi el trasllat, conviuen els dos programes i es poden utilitzar conjuntament.

Com es pot veure en la figura 1, les aplicacions compten amb mòduls centrats en els diferents aspectes necessaris en la gestió de les construccions. En el cas d'aquest treball, les tasques es realitzen en el mòdul de Pressupost (PRE) i en el de Certificació (CER) a vegades anomenat també com a mòdul de Seguiment. L'abast del projecte es troba en un apartat a continuació, però a grans trets el projecte vol aconseguir millorar el rendiment i el temps de resposta en les funcionalitats del mòdul per tal que els clients d'escriptori tinguin una experiència d'usuari el més similar possible en la versió cloud.

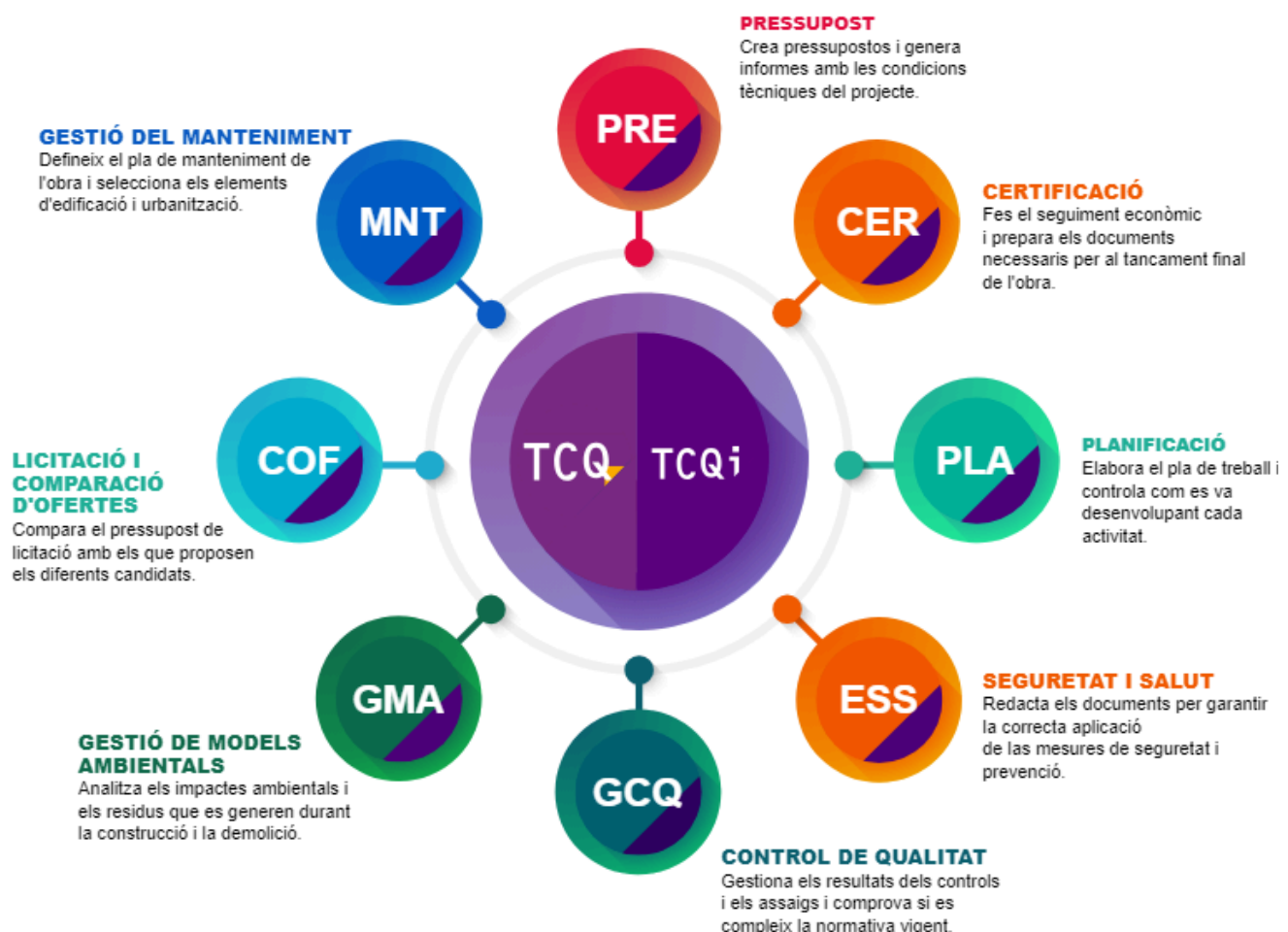


Figura 1: Esquema dels mòduls de TCQ i TCQi. Font: ITec

11. Especificació

En aquest apartat del treball s'especifica el comportament dels mòduls del treball de forma genèrica, per poder veure a continuació les tasques modificades durant el projecte per tal de millorar-les. Primerament, es mostrarà l'estructura del sistema a través d'un diagrama de classes. A continuació quedaran llistats i explicats els casos d'ús particulars per les diferents funcionalitats compreses dins les optimitzacions del projecte, detallant finalment les accions que pot fer l'usuari amb un model de comportament.

11.1. Diagrama de casos d'ús

Per començar amb l'especificació, ens cal deixar definits els casos d'ús [17] que apareixen en les funcionalitats optimitzades durant el projecte, per tal de poder veure els requisits contextualitzats en el sistema i veure la interacció dels possibles diferents actors. Per fer-ho S'ha utilitzat un diagrama realitzat utilitzant la notació gràfica UML (Unified Modeling Language) [18] per poder representar les funcionalitats del sistema des del punt de vista dels actors involucrats.

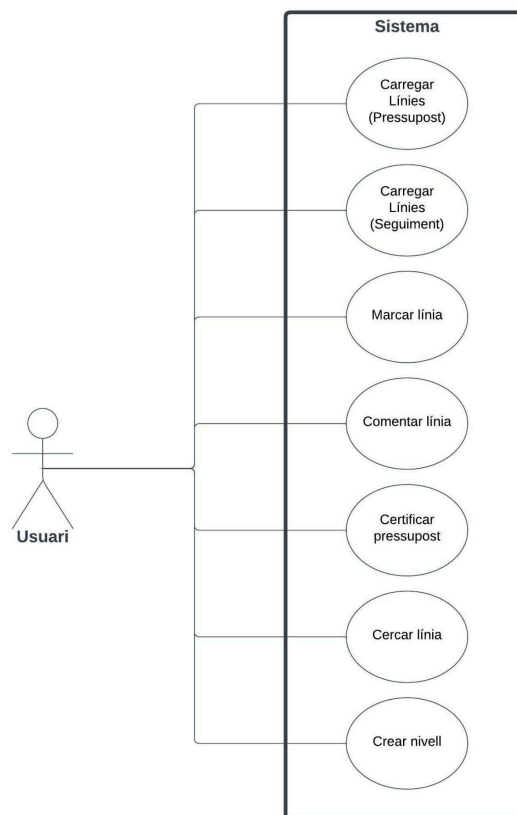


Figura 3: Diagrama de casos d'ús. Font: Elaboració pròpia.

Usuari: L'usuari és l'actor implicat que treu benefici directe de l'ús del producte. En l'aplicació TCQi l'usuari accedeix a través d'un compte per poder fer les accions que consideri. Segons el tipus de compte i les llicències que tingui activades, l'aplicació reconeix els permisos i li permet accedir o portar a cap les accions.

Sistema: El sistema és l'entorn on els casos d'ús s'efectuen. En el cas d'aquest projecte és el programa TCQi, específicament els mòduls de Pressupost (PRE) i Seguiment o Certificació (CER). Més endavant s'especifica com estan organitzats i com els hi arriba la informació de les accions dels usuaris.

Tal com es pot observar en el diagrama anterior, hi ha diverses accions que l'actor pot prendre. Aquestes són les interaccions que s'han vist optimitzades per l'autor, dins dels mòduls es troben d'altres que al estar realitzades per altres companys no han estat afegides en aquest treball. A continuació es descriuen els casos d'ús mostrats al diagrama.

11.2. Model conceptual

Mitjançant el diagrama de classes afegit a continuació, s'explicarà com està organitzada l'estructura de TCQi. Com es tracta d'un programa complex i de grans dimensions, per tal d'adequar-se al projecte, únicament s'han tingut en compte les classes que afecten directament les tasques dutes a terme, però en realitat a les aplicacions de pressupost i seguiment hi tenen relació moltes més classes de les representades. De forma semblant a les classes, s'han obviat aquells atributs que no tenen relació directa. Únicament apareixen aquells atributs que intervenien en els casos de les funcions del projecte. Dins el sistema de l'empresa hi ha molts més atributs, i per això en el diagrama apareixen punts suspensius com a indicadors d'aquesta situació.

Un diagrama de classes té els següents components:

Classe: Abstracció d'un conjunt d'objectes del sistema

Atribut: Característica o propietat concreta de la classe.

Relació: Connexió que defineix la interacció que tenen les classes.

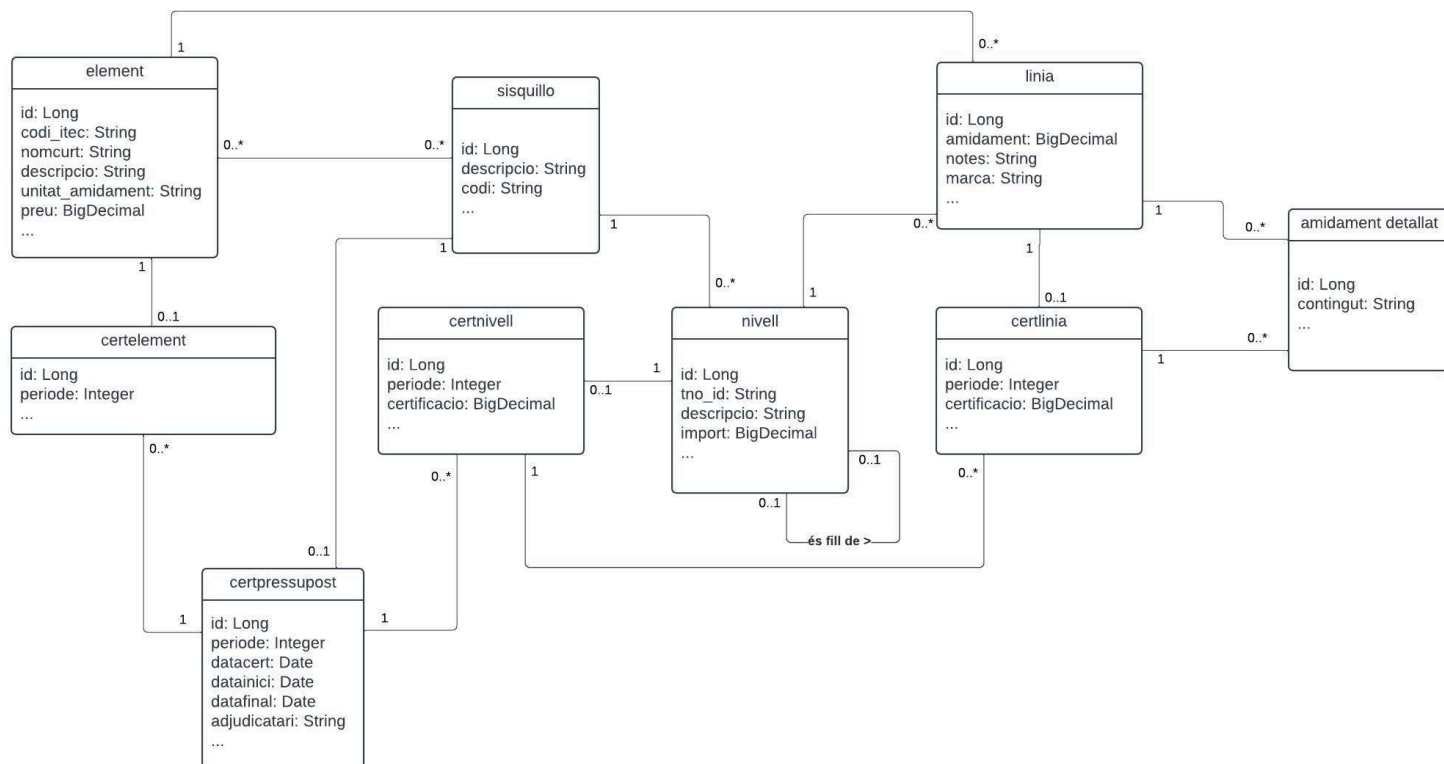


Figura 4: Model conceptual del sistema. Font: Elaboració pròpia.

Un cop ja definida l'estructura del sistema dins del projecte i la seva representació lògica, veiem a continuació com s'apliquen les restriccions d'integritat per tal que el diagrama anterior i el sistema tinguin coherència. A més a més, s'explica com es determinen cadascuna de les classes en més detall i quina representació tenen dins les tasques del projecte.

11.2.1. Restriccions d'integritat

Les restriccions d'integritat són un seguit d'afirmacions o normes que el sistema ha de seguir per funcionar de manera correcta. Aquestes restriccions, com no tenen forma de quedar representades gràficament s'afegeixen de forma escrita. En aquest apartat s'explicaran quines són les restriccions d'integritat de la Figura 3.

Claus externes:

(sisquillo, id), (certpressupost, id), (amidament detallat, id), (linia, id), (nivell, id), (element, id), (certlinia, id), (certnivell, id), (certelement, id).

RT1: A la classe certpressupost la datainici ha de ser anterior a la datafinal.

RT2: A la classe certpressupost la datacert ha de ser posterior o igual a la datafinal.

RT3: Una instància d'amidament detallat no pot estar relacionada amb certlinia si no ho està amb línia.

RT4: Si amidament detallat està relacionat amb certlinia els atributs id_linia de totes dues classes han de coincidir.

RT5: Tots els atributs periode han de tenir el mateix valor.

RT6: L'atribut periode no pot prendre un valor negatiu.

11.2.2. Descripció de les classes

Profunditzem en les diferents classes que conformen el diagrama anterior i els atributs de cada classe.

- **Sisquillo:** Representa l'obra o procés constructiu sobre la qual el client efectua el pressupost, la certificació o altres serveis no treballats durant el transcurs del projecte. Conté dades genèriques usades en els diferents mòduls de TCQi.
 - id: Identificador únic de l'objecte.
 - descripcio: Text que descriu l'obra.
 - codi: Combinació de caràcters i números únic.
- **Cert Pressupost:** És un model que guarda la informació de les obres ja pressupostades i que estan en seguiment, en el cas del projecte s'ha centrat en treballar amb les dates d'inici i final de les certificacions i el període en el qual es troba.
 - id: Identificador únic de l'objecte.
 - periode: Número que assenyalava en quina secció del seguiment es troba la instància.
 - datacert: Data en la qual se certifica
 - datainici: Data en la qual inicia el seguiment.
 - datafinal: Data en la qual finalitza el seguiment.
 - adjudicatari: Nom de l'encarregat d'adjudicar el seguiment al pressupost de l'obra.
- **Nivell:** El nivell és el nom que agafen els nodes de l'arbre dins l'obra en el mòdul de pressupost. En el cas d'aquest projecte únicament necessitem el 'codi', la descripció i l'import per les funcionalitats realitzades.
 - id: Identificador únic de l'objecte.
 - tno_id: Combinació de lletres i números única. Funciona com a codi identificador.
 - descripcio: Text que descriu el nivell.
 - import: Cost calculat a través del preu dels continguts del nivell.

- **Cert Nivell:** El nivell certificat és l'equivalent de nivell però en el mòdul de seguiment. Se sosté d'informació del mateix nivell i afegeix dades pels càlculs segons el període i les certificacions acumulades.
 - id: Identificador únic de l'objecte.
 - periode: Número que assenyala en quina secció del seguiment es troba la instància.
 - certificacio: Valor de la quantitat certificada actualment a la instància.

- **Línia:** És una classe que recull la informació dels nodes fulla de l'arbre de l'obra. La línia representa un conjunt de partides o elements. Per aquest projecte s'ha necessitat la informació que contenen de mides, el color o les notes.
 - id: Identificador únic de l'objecte
 - amidament: Valor per mesurar la quantitat, volum, mida, unitats... de la línia.
 - notes: Text afegit per l'usuari per donar context a la línia.
 - marca: Color seleccionat per l'usuari per diferenciar la línia.

- **Cert Línia:** La línia certificada és l'equivalent de la línia en el mòdul de seguiment. Reaprofita el contingut de la línia a la qual fa referència i afegeix informació i càlculs utilitzats per la certificació.
 - id: Identificador únic de l'objecte.
 - periode: Número que assenyala en quina secció del seguiment es troba la instància.
 - certificacio: Valor de la quantitat certificada actualment a la instància.

- **Element:** L'element és la representació de les accions físiques que els obrers han de fer al dia a dia de l'obra a pressupostar. Dins el model conté atributs per descriure i que l'usuari entengui de quina acció tracta i conté també informació del cost i de la manera de mesurar que té.
 - id: Identificador únic de l'objecte.
 - codi_itec: Combinació de lletres i números únic. Funciona com a codi identificador.
 - nomcurt: El nom de l'element que es mostrarà a l'aplicació.
 - descripcio: Text que explica com és l'element.
 - unitat_amidament: Unitats en les quals es mesura l'element.
 - preu: Cost de mercat de l'element.

- **Cert Element:** És una classe que es recolza de l'anterior, s'utilitza en el mòdul de seguiment i afegeix la informació necessària per certificar les quantitats mesurades segons el període en el qual el seguiment es troba.
 - id: Identificador únic de l'objecte.
 - periode: Número que assenyala en quina secció del seguiment es troba la instància.
- **Amidament Detallat:** És una classe que recull les fórmules necessàries per justificar i detallar com s'efectuen els càlculs de les mides de la línia a la qual relaciona.
 - id: Identificador únic de l'objecte.
 - contingut: Fórmula específica de la instància d'amidament.

11.3. Descripció de casos d'ús

Els casos d'ús que hem vist a l'inici de la secció 11, queden descrits a través de taules amb la seva informació corresponent. En aquest apartat s'especifica el nom del cas d'ús, l'actor principal, les precondicions que ha de complir, el disparador que activa el cas d'ús, l'escenari principal d'èxit i els possibles escenaris alternatius i les accions corresponents per respondre al problema.

Cas d'ús	Carregar línies de pressupost	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema.		
Disparador	- L'usuari vol accedir a la informació i a les partides d'una línia en el mòdul de pressupost.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQi Pressupost. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari navega clicant a un nivell de l'arbre de l'obra. 4. Si es tracta d'un nivell fulla el sistema mostra les línies que conté.			
Escenaris alternatius			
4.a L'aplicació indica que ha sorgit un error en carregar les línies. 4.a.1 No apareixen línies ni la seva informació 4.a.2 Es notifica a l'usuari que hi ha hagut un error en l'accés a la informació.			

Taula 10: Cas d'ús - Carregar línies de pressupost. Font: Elaboració pròpia.

Cas d'ús	Carregar línies de seguiment	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema.		
Disparador	- L'usuari vol accedir a la informació i a les partides d'una línia en el mòdul de seguiment.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQ i Certificació. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari navega clicant a un nivell de l'arbre de l'obra. 4. Si es tracta d'un nivell fulla el sistema mostra les línies que conté.			
Escenaris alternatius			
4.a L'aplicació indica que ha sorgit un error en carregar les línies. 4.a.1 No apareixen línies ni la seva informació 4.a.2 Es notifica a l'usuari que hi ha hagut un error en l'accés a la informació.			

Taula 11: Cas d'ús - Carregar línies de seguiment. Font: Elaboració pròpia.

Cas d'ús	Marcar línia	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'escollir un color per marcar la línia.		
Disparador	- L'usuari vol diferenciar la línia escollida canviant-la de color.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQ i Pressupost. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari navega clicant a un nivell de l'arbre de l'obra. 4. Si es tracta d'un nivell fulla el sistema mostra les línies que conté. 5. L'usuari clica a la icona de configuració que apareix a la línia que vol marcar per desplegar el menú d'accions. 6. L'usuari clica sobre l'acció 'Marcar línia'. 7. El sistema obre un petit menú amb diferents selectors de colors. 8. L'usuari escull el color que vol i pren el botó 'Seleccionar'. 9. El sistema torna a carregar la línia amb les modificacions realitzades.			
Escenaris alternatius			
8.a L'aplicació indica que ha sorgit un error en marcar la línia 8.a.1 No canvia el color de fons de la línia.			

Taula 12: Cas d'ús - Marcar línia. Font: Elaboració pròpia.

Cas d'ús	Comentar línia	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'escriure un comentari a la línia.		
Disparador	- L'usuari vol deixar un comentari a la línia seleccionada.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQI Pressupost. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari navega clicant a un nivell de l'arbre de l'obra. 4. Si es tracta d'un nivell fulla el sistema mostra les línies que conté. 5. L'usuari clica a la icona del full que apareix a la línia que vol comentar. 6. El sistema obre un petit menú amb un camp per escriure. 7. L'usuari escriu el comentari i pren el botó 'Guardar'. 8. El sistema torna a carregar la línia amb les modificacions realitzades.			
Escenaris alternatius			
7.a L'aplicació indica que ha sorgit un error en comentar la línia			
7.a.1 No canvia la icona del full, ni apareix el comentari en passar el ratolí.			

Taula 13: Cas d'ús - Comentar línia. Font: Elaboració pròpia.

Cas d'ús	Certificar pressupost	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema. - L'obra que es vol certificar no té dades de seguiment. - L'obra que es vol certificar té dades de pressupost.		
Disparador	- L'usuari vol iniciar el seguiment d'un pressupost finalitzat.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQI Certificació en una obra que no tingui el seguiment començat i tingui informació de pressupost. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari omple el formulari amb les dades pertinents. 4. L'usuari marca l'opció de 'Guardar'. 5. El sistema valida que la informació del formulari és correcta. 6. El sistema genera el seguiment de l'obra.			
Escenaris alternatius			
5.a L'aplicació indica que ha sorgit un error en la validació del formulari 5.a.1 La data d'inici és posterior a la de tancament. 5.a.2 La data de tancament és posterior a la data de certificació. 5.a El sistema informa de l'error i sol·licita dates correctes.			

Taula 14: Cas d'ús - Certificar pressupost. Font: Elaboració pròpia.

Cas d'ús	Cercar línia	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'introduir algun caràcter o paraula.		
Disparador	- L'usuari vol trobar una línia a través d'algun caràcter o paraula.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQí Pressupost. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari clica a l'apartat amb la icona de la lupa que posa 'CERCA'. 4. L'usuari introdueix el valor de la línia que vol trobar. 5. El sistema retorna una llista de les línies que contenen el text introduït en algun atribut.			
Escenaris alternatius			
-			

Taula 15: Cas d'ús - Cercar línia. Font: Elaboració pròpia.

Cas d'ús	Crear nivell	Actor	Usuari
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'omplir el formulari amb els valors del nivell corresponents.		
Disparador	- L'usuari vol afegir un nou nivell al node de l'arbre escollit.		
Escenari principal d'èxit			
1. L'usuari inicia sessió a l'aplicació de TCQí Pressupost. 2. El sistema valida les dades i li dona accés al mòdul. 3. L'usuari navega clicant a un nivell de l'arbre de l'obra. 4. Si el nivell no té línies, el sistema mostra una icona amb el símbol '+ '. 5. L'usuari prem la icona. 6. L'usuari posa un codi i un nom i prem l'opció de 'Guardar'. 7. El sistema crea el nivell i l'afegeix a l'arbre.			
Escenaris alternatius			
6.a L'aplicació indica que ha sorgit un error en la validació del formulari			
6.a.1 El codi introduït ja existeix a l'obra.			

Taula 16: Cas d'ús - Crear nivell. Font: Elaboració pròpia.

11.4. Model de comportament

En aquest apartat del document es mostra el model de comportament. Per cada cas d'ús descrit anteriorment, el model detalla la interacció de l'usuari amb el sistema. Individualment, mostra la petició de l'usuari o client i la resposta que obté per part del sistema. D'igual forma que en els casos d'ús els elements que participen en el model són l'actor i el sistema. L'actor referència a l'usuari com a persona implicada en realitzar l'acció. El sistema referència a tot el programari intern de l'aplicació.

Carregar línies de pressupost

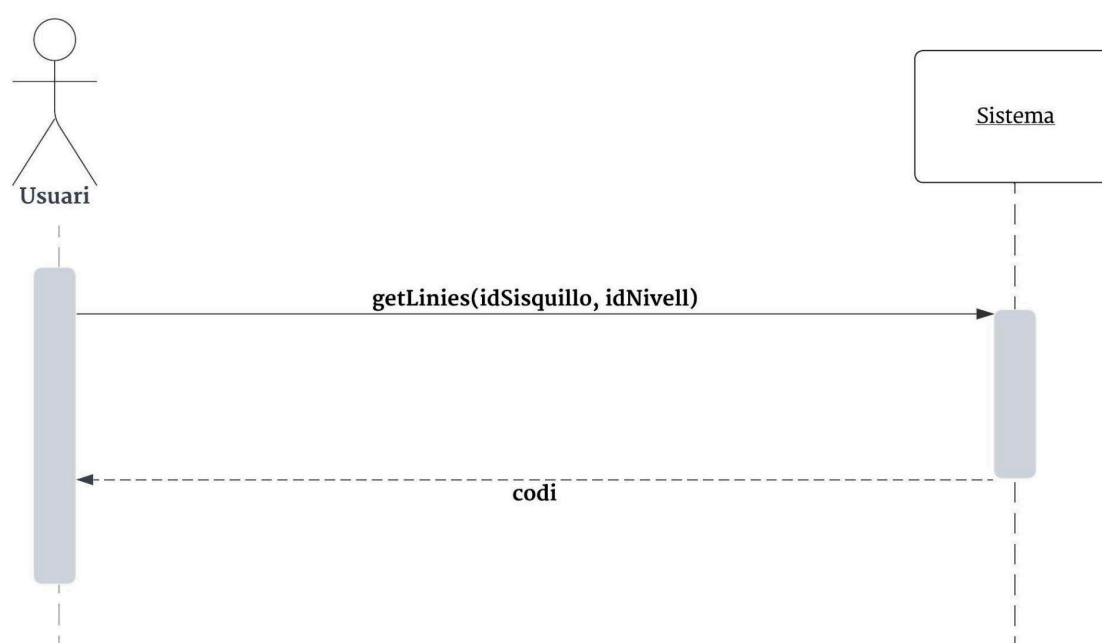


Figura 5: Diagrama de comportament – Carregar línies de pressupost. Font: Elaboració pròpia.

getLinies(idSisquillo: Long, idNivell: Long)	
Precondicions	L'usuari està identificat en el sistema
Postcondicions	Es llisten a l'aplicació totes les línies del nivell.
Body	Es retorna el codi d'acceptació si s'han pogut aconseguir les dades correctament.

Taula 17: Comportament – Carregar línies de pressupost. Font: Elaboració pròpia.

Carregar línies de seguiment

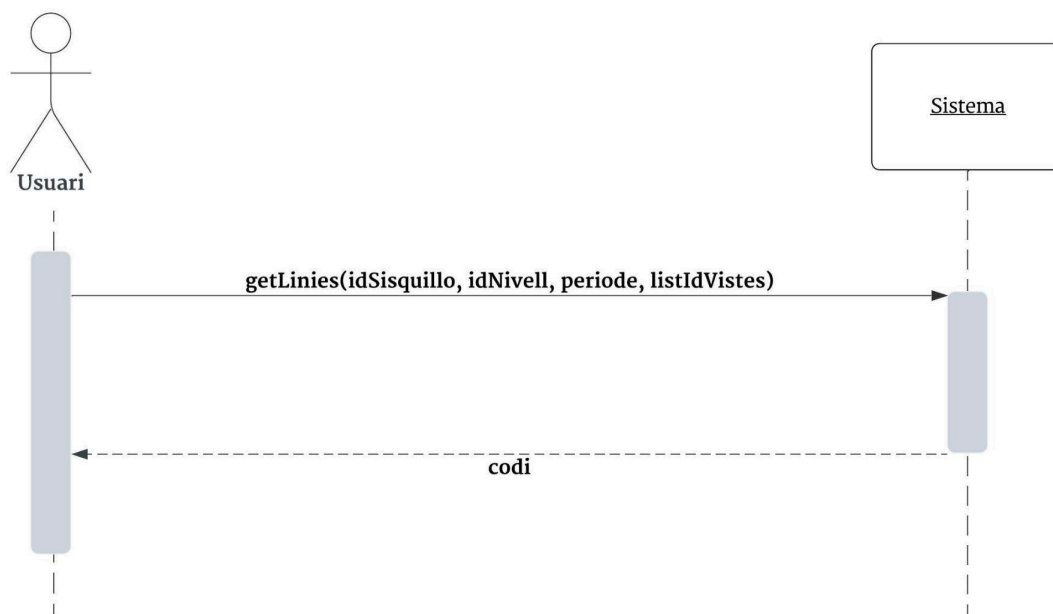


Figura 6: Diagrama de comportament - Carregar línies de seguiment. Font: Elaboració pròpia.

getLinies(idSisquillo: Long, idNivell: Long, periode: Integer, listIdVistes: List<Long>)	
Precondicions	- L'usuari està identificat en el sistema
Postcondicions	- Es llisten a l'aplicació totes les línies del nivell.
Body	- Es retorna el codi d'acceptació si s'han pogut aconseguir les dades correctament.

Taula 18: Comportament - Carregar línies de seguiment. Font: Elaboració pròpia.

Marcar línia

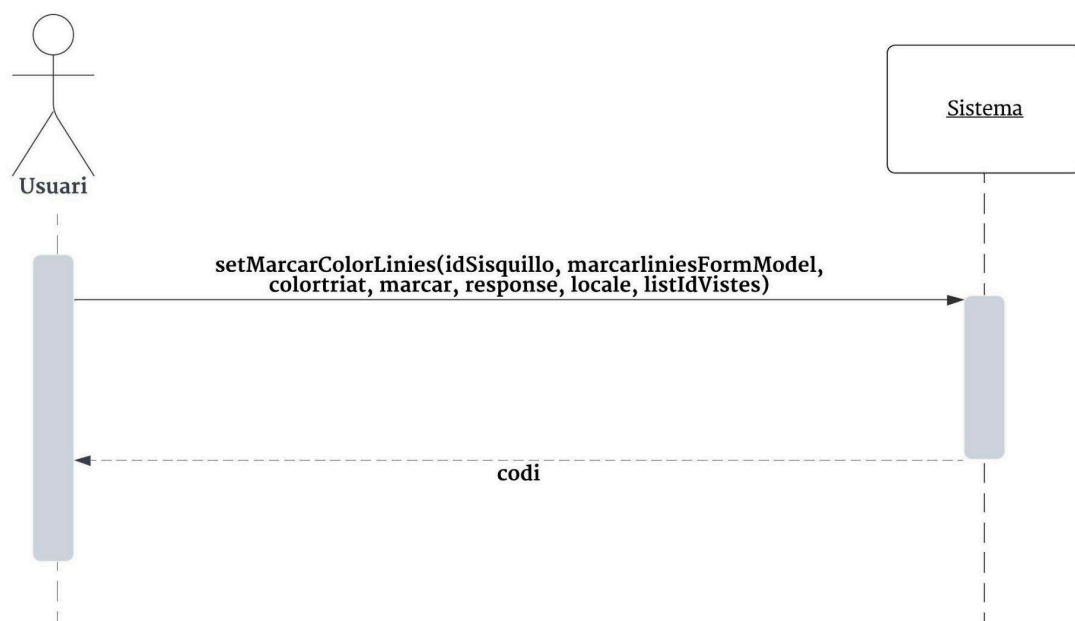


Figura 7: Diagrama de comportament - Marcar línea. Font: Elaboració pròpia.

setMarcarColorLinies(idSisquillo: Long, marcarliniesFormModel: NivellLiniesDataModel, colortriat: String, marcar: Boolean, response: Response, locale: Locale, listIdVistes: List<Long>)	
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'escollir un color per marcar la línia.
Postcondicions	- Es veu la línia amb el color modificat.
Body	- Es retorna el codi d'acceptació si s'han pogut modificar les dades correctament.

Taula 19: Comportament - Marcar línea. Font: Elaboració pròpia.

Comentar línia

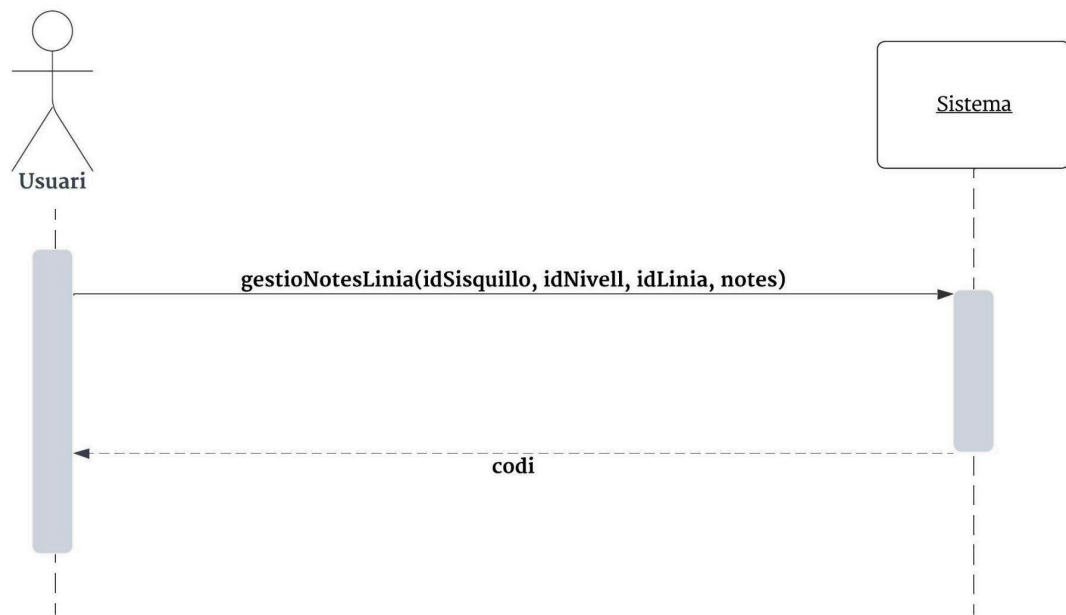


Figura 8: Diagrama de comportament - Comentar línia. Font: Elaboració pròpia.

gestioNotesLinia(idSisquillo: Long, idNivell: Long, idLinia: Long, notes: String)	
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'escriure un comentari a la línia.
Postcondicions	- Es veu la icona del full omplert i mostra el comentari en passar el ratolí per damunt.
Body	- Es retorna el codi d'acceptació si s'han pogut modificar les dades correctament.

Taula 20: Comportament - Comentar línia. Font: Elaboració pròpia.

Certificar pressupost

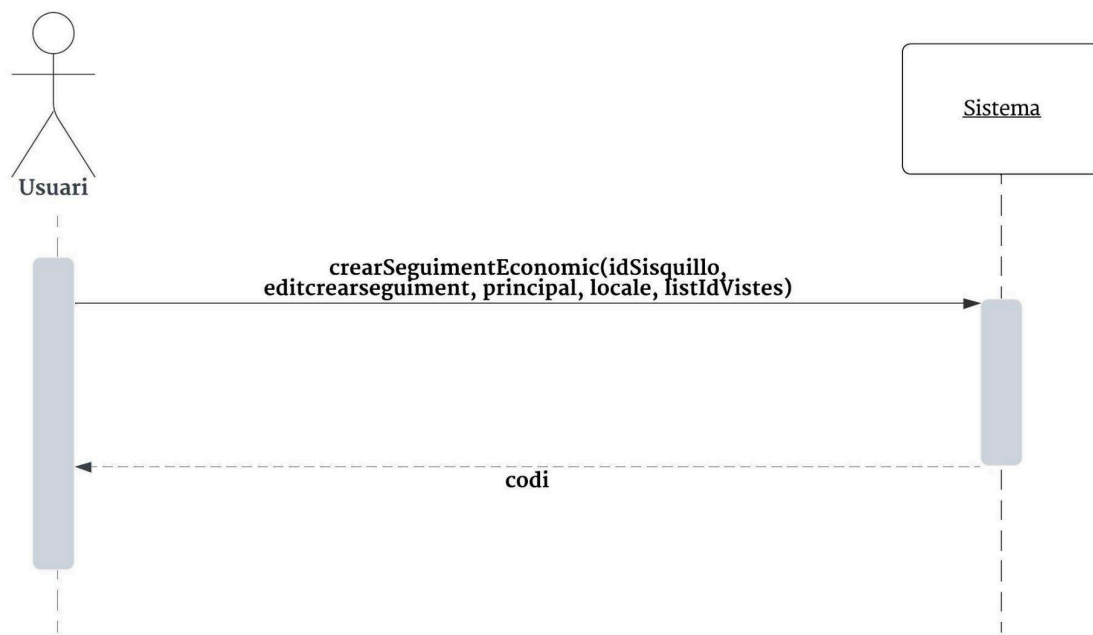


Figura 9: Diagrama de comportament - Certificar pressupost. Font: Elaboració pròpia.

crearSeguimentEconomic(idSisquillo: Long, editcrearseguiment: EditCrearSeguiment, principal: Principal, locale: Locale, listIdVistes: List<Long>)	
Precondicions	- L'usuari està identificat en el sistema. - L'obra que es vol certificar no té dades de seguiment. - L'obra que es vol certificar té dades de pressupost.
Postcondicions	- S'obre el mòdul de Certificació amb la informació de l'obra preparada en el mòdul de Pressupost.
Body	- Es retorna el codi d'acceptació si s'han pogut afegir les dades correctament.

Taula 21: Comportament - Certificar pressupost. Font: Elaboració pròpia.

Cercar línia

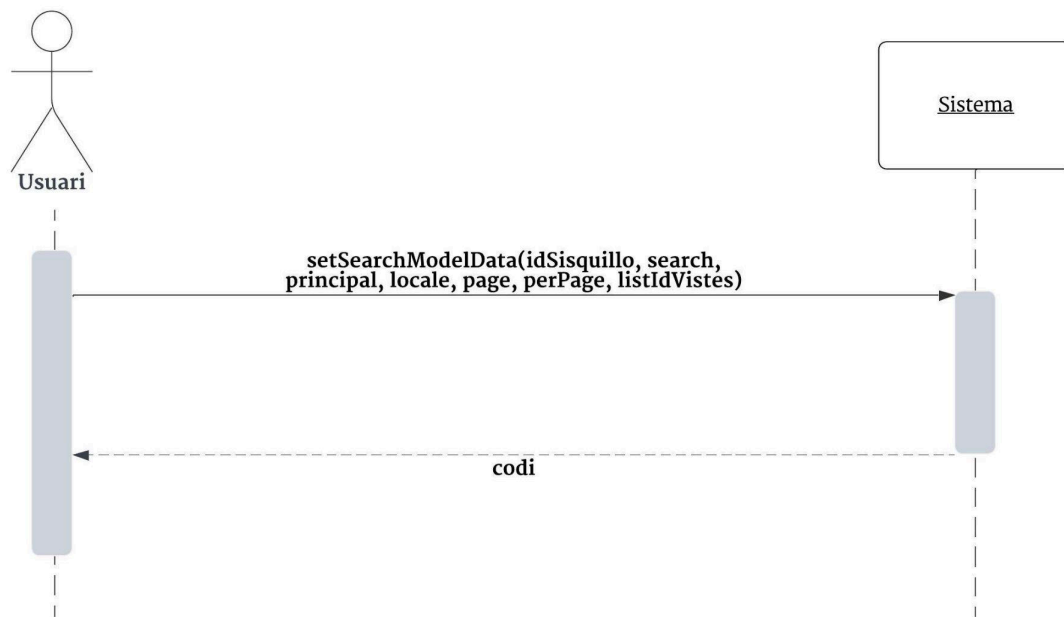


Figura 10: Diagrama de comportament - Cercar línia. Font: Elaboració pròpia.

setSearchModelData(idSisquillo: Long, search: String, principal: Principal, locale: Locale, page: Integer, perPage: Integer, listIdVistes: List<Long>)	
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'introduir algun caràcter o paraula.
Postcondicions	- Es llisten totes les línies en les quals aparegui el text introduït per l'usuari.
Body	- Es retorna el codi d'acceptació si s'han pogut aconseguir les dades correctament.

Taula 22: Comportament - Cercar línia. Font: Elaboració pròpia.

Crear nivell

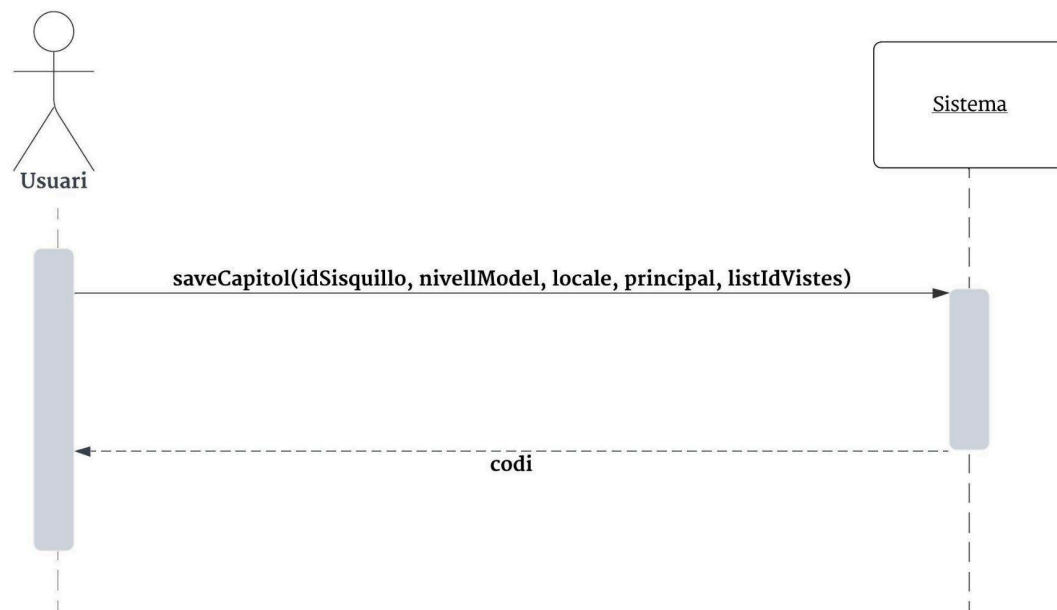


Figura 11: Diagrama de comportament - Crear nivell. Font: Elaboració pròpia.

saveCapitol(idSisquillo: Long, nivellModel: NivellModel, locale: Locale, principal: Principal, listIdVistes: List<Long>)	
Precondicions	- L'usuari està identificat en el sistema. - L'usuari ha d'omplir el formulari amb els valors del nivell corresponents.
Postcondicions	- S'afegeix el nou nivell a l'arbre de l'obra.
Body	- Es retorna el codi d'acceptació si s'han pogut afegir les noves dades correctament.

Taula 23: Comportament - Crear nivell. Font: Elaboració pròpia.

12. Disseny

Després d'explicar l'especificació del sistema format per les tasques optimitzades durant el projecte i el funcionament d'aquestes, cal entendre com està internament dissenyada l'aplicació TCQI, quins components la formen i com es traslladen la informació per tal de fer funcionar el programa. En aquest apartat es llistaran les tecnologies emprades durant el desenvolupament del codi i les tecnologies necessàries per fer arribar TCQI a tots els seus clients. També s'explicarà quin és el disseny arquitectònic de l'aplicació i quines modificacions s'han afegit per optimitzar els seus mòduls.

12.1. Arquitectura

L'aplicació TCQI i els diferents mòduls que ofereix, estan compostos internament per tres grans blocs. Per poder entendre els canvis aplicats a Pressupost i Certificació per millorar el seu rendiment, cal conèixer primer com està definida l'estructura de l'aplicació. En aquest apartat veurem quin són aquests tres blocs, com es comparteixen entre ells la informació i com queda l'arquitectura definida abans i després d'introduir les millores.

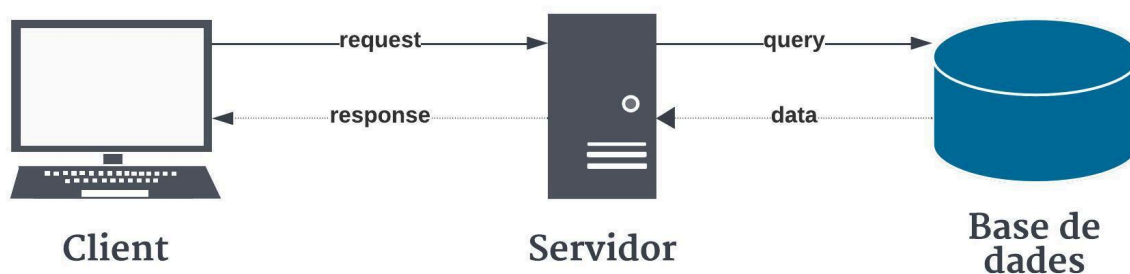


Figura 12: Components de l'aplicació i com interactuen. Font: Elaboració pròpia.

A la figura es pot observar els tres components principals de l'aplicació i quin tipus de comunicació tenen entre ells. A continuació es descriuen els blocs:

- Client (*frontend*)

En aquest punt de l'aplicació és on trobem desenvolupada tota la lògica dels components visuals de la pàgina web. Aquí estan definides totes les diferents accions que pugui fer l'usuari amb els elements que apareixen a les vistes de l'aplicació. La majoria d'aquestes interaccions es reenvien cap al servidor o *backend* perquè el sistema les gestioni, faci una sol·licitud al servidor de l'API [19] per poder retornar les dades necessàries. En el cas particular d'aquest projecte no s'ha dissenyat ni modificat res amb relació a la part de client de l'aplicació. Per aquest

motiu el treball no profunditzarà en aquest component, però igualment ha estat necessari conèixer i entendre el seu funcionament.

- **Servidor (*backend*)**

La part de servidor serveix d'intermediari entre les dades de l'obra i el que veu i acciona l'usuari. Aquí està desenvolupada tota la lògica darrere les operacions i els càlculs que es necessitin realitzar per les diverses funcionalitats oferides a l'aplicació. És un bloc que està format pel servidor físic que emmagatzema l'API del programa. El gruix del projecte ha recaigut en aplicar modificacions en aquest apartat, per això en els següents apartats s'entrarà en detall sobre l'arquitectura del *backend* i en el seu estat previ i posterior a les implementacions de les millores.

- **Base de dades**

Aquest element constitueix la base on s'emmagatzemen totes les dades essencials per al funcionament de l'aplicació. La seva funció també implica establir connexions lògiques entre aquestes dades, amb l'aplicació seguint aquesta estructura per proporcionar diverses funcionalitats. A més, aquest component manté una connexió directa amb el *backend* de l'aplicació, encarregant-se de respondre a les crides provinents d'aquesta secció per subministrar, eliminar o modificar les dades necessàries.

12.1.1. Backend

Fins que es va iniciar el projecte de l'optimització de Pressupost i Certificació, l'estructura de la part de servidor era comú en tots els mòduls de TCQI. No obstant això, el canvi simplement el trobem en la gestió de les entitats i els accessos a la base de dades que utilitza per recollir la informació de les diferents entitats. Ara es realitza a través de tecnologies diferents segons si els mòduls estan sent optimitzats o no. Pel que fa a les funcions i tasques que duu a terme no hi ha cap diferència, en tots dos casos el servidor s'encarrega de respondre a les peticions fetes per l'usuari des del *frontend*. A més estableix la connexió amb la base de dades fent ús de o bé la tecnologia Hibernate [\[20\]](#) o bé la tecnologia Blaze, depenent el mòdul que tracti.

El que és comú en tots els mòduls és l'estructura que segueix el *backend*. Per mantenir les diferents parts del codi organitzades i per facilitar la feina, la part de servidor segueix una estructura de tres nivells basada en el patró model vista controlador (MVC) [\[21\]](#). En el cas d'aquest bloc de l'aplicació, els models del patró fan referència a les dades recollides a la capa de persistència de la base de dades. Com no conté interfícies d'usuari, les vistes del patró són totes les comunicacions JSON [\[22\]](#) que s'envien cap al bloc de *frontend*. I pel que fa als controladors, queden explicats en profunditat a continuació.

- **Controller**

En aquesta fase, la part del *backend* de l'aplicació assumeix la responsabilitat de processar les sol·licituds procedents del *frontend*. Aquestes sol·licituds són rebudes mitjançant l'ús del *framework* [23] Spring i arriben al *backend* a través d'una URL específica. Aquesta URL està integrada en un dels controladors, que és responsable de gestionar i respondre a la sol·licitud. Un cop les dades han estat adequadament processades i s'ha completat l'acció sol·licitada per l'usuari, el *backend* envia una resposta, o en cas de qualsevol error durant el procés, emet la resposta d'error pertinent.

- **Business**

En aquesta etapa, s'executen totes les operacions relacionades amb la lògica de negoci. Després que el controlador hagi rebut les sol·licituds, aquest component transmet la tasca a la capa de negoci, la qual estableix la comunicació amb la capa de persistència per dur a terme l'acció de l'usuari. En aquest nivell de negoci, sovint s'implementen tots els algorismes necessaris per mantenir la coherència de l'aplicació i assegurar-ne el funcionament adequat.

- **Persistence DAO**

Aquesta constitueix la capa fonamental del sistema, encarregada de realitzar els accessos a la base de dades i ajustar-la en funció de les accions de l'usuari. L'acrònim DAO (Data Access Object) representa un patró utilitzat per declarar els objectes de persistència, com ara les taules de la base de dades, amb l'objectiu d'assolir desacoblament. Les operacions potencials sobre les taules de la base de dades inclouen creació, modificació, lectura i esborrat. En aquesta capa, les dades es converteixen de taules a objectes de persistència, que es representen com a classes en el *backend* de l'aplicació. Posteriorment, les dades es retornen a la capa de negoci perquè se'n faci la gestió segons les funcionalitats requerides. Cal destacar la importància de minimitzar les crides a la base de dades en aquesta capa, ja que aquesta pràctica pot impactar en el rendiment de l'aplicació.

Aquests controladors descrits són els que feia servir tota l'aplicació abans del projecte, i els que fan servir encara la resta de mòduls. Pel que fa a Pressupost i Certificació s'ha canviat com es gestionen els objectes de persistència. Queda enrere l'ús dels DAO i a través de Blaze es troba una solució. Es va veure que hi havia moltes funcionalitats on per seguir l'estructura creada amb els DAO, l'aplicació feia grans quantitats d'accessos o feia accessos on extreia grans quantitats d'informació quan únicament utilitzava o necessitava una petita part. Per aquest motiu es va escollir l'alternativa on a través dels components que s'explica a continuació s'ha pogut trobar la solució per aquests casos.

- Vistes Blaze

Aquest component de Blaze Persistence representa una capa significativa en el sistema, destinada a proporcionar una abstracció potent i eficient per a l'accés a dades. Aquesta característica és fonamental per optimitzar les consultes a la base de dades, ja que les Vistes Blaze permeten definir vistes específiques dels objectes de persistència. Amb l'ús d'aquesta capa, es pot reduir la quantitat de dades recuperades de la base de dades, perquè aquestes vistes permeten especificar només les dades necessàries per a una operació o funcionalitat concreta. Això no només millora el rendiment de l'aplicació, sinó que també contribueix a la claredat del codi, ja que es pot treballar amb conjunts de dades més petits i específics

- Repositoris Blaze

Aquesta funcionalitat permet als desenvolupadors interactuar amb la capa de persistència de manera més simplificada i coherent. Els Repositoris Blaze ofereixen una interfície que facilita l'accés i la manipulació de les dades emmagatzemades, proporcionant mètodes predefinits per a operacions com la lectura, escriptura, eliminació i altres funcions comunes. Això no només redueix la complexitat del codi, sinó que també fomenta una major reutilització de codi i mantenibilitat. Una característica distintiva és la seva integració amb el DSL (Llenguatge Específic del Domini) de Blaze, que facilita la creació de funcions de consulta. Amb aquest DSL, els desenvolupadors poden expressar consultes amb una sintaxi lògica i orientada al domini, simplificant així la creació de consultes complexes. Aquest abordatge permet una millor llegibilitat del codi i una expressió més eficaç de les necessitats de consulta, afegint valor a la capa de persistència de l'aplicació. En conjunt amb les Vistes Blaze, els Blaze Repositoris proporcionen una solució completa i optimitzada per a la gestió de dades en aplicacions complexes i de gran quantitat d'informació com és el cas de TCQj.

Aquesta arquitectura de diferents nivells en la part del servidor assegura que hi hagi una separació de les responsabilitats dins les classes, fent que el codi sigui més depurable i més fàcil d'entendre. A més, aquesta organització facilita la possibilitat d'escalar, afegint nous recursos si les capes ho necessiten. Com ha estat el cas per aquest projecte. Les diferents capes ens asseguren també més seguretat i facilitat per fer tests separats, identificar errors ràpidament fent que l'aplicació sigui més consistent.

12.2. Tecnologies utilitzades

Anteriorment, a la secció 6.2.3 del treball, s'han llistat els diferents recursos informàtics usats per portar a terme aquest projecte. En aquesta secció, en canvi, es mostren les tecnologies específicament necessàries per a desenvolupar i mantenir l'aplicació. Aquestes són les següents:

Java Platform Standard Edition (Java SE)

És una col·lecció d'APIs basades en el llenguatge de programació Java, destinades al desenvolupament del codi de l'aplicació en la part del servidor (*backend*). Aquesta plataforma ofereix totes les llibreries fonamentals de Java, a més de proporcionar classes d'alt nivell per a funcionalitats com seguretat, accés a bases de dades i treball en xarxa, entre d'altres. Amb una arquitectura apta per a sistemes multicapes, Java SE permet als desenvolupadors gestionar aplicacions empresarials complexes i actuar amb servidors d'aplicacions.

Spring

Es tracta d'un *framework* àmpliament utilitzat per facilitar la codificació en el *backend* d'aplicacions i pàgines web. Algunes de les seves característiques principals inclouen la inversió de control, que permet una millor gestió de dependències i l'organització del codi; la programació orientada a aspectes, que facilita la separació de preocupacions i millora la modularitat del codi; i una atenció especial a la seguretat en el desenvolupament. Dins dels mòduls de TCQi, Spring és usat com a *framework* en la part del servidor, destacant-se com un dels més reconeguts per la seva flexibilitat i facilitat d'ús en el desenvolupament d'aplicacions Java.

Hibernate

Aquesta tecnologia és una eina ORM (Mapeig Objecte-Relacional) que es fa servir per simplificar la comunicació entre aplicacions Java i bases de dades relacionals. Aquesta tecnologia ofereix un mecanisme eficient per mapejar objectes Java a taules de bases de dades i viceversa, eliminant la necessitat de gestionar manualment les consultes SQL. A l'ITeC, s'utilitza en moltes aplicacions de l'empresa amb l'objectiu d'optimitzar el desenvolupament i estalviar temps en la interacció entre la capa de persistència i la base de dades. Hibernate proporciona una abstracció que permet als desenvolupadors centrar-se més en la lògica de l'aplicació i menys en les complexitats de la gestió de dades en l'àmbit de base de dades.

Blaze Persistence

Blaze és una eina ORM avançada, que prioritza l'alt rendiment en el mapeig objecte-relacional. Està especialment dissenyada per interactuar amb bases de dades relacionals, Blaze ofereix una solució eficient per convertir objectes Java en taules de bases de dades i viceversa. A l'ITeC, aquesta tecnologia s'ha convertit en la solució escollida per abordar les consultes que estaven donant problemes de rendiment, usant vistes i repositoris de Blaze per optimitzar els accessos a les bases de dades. Aquesta implementació ens ha permès als desenvolupadors mitigar la complexitat de les operacions de persistència i millorar significativament el rendiment dels accessos a les dades.

Jenkins

És un servidor d'automatització de codi obert que es fa servir a l'ITeC per aconseguir la pràctica de la integració continuada i entrega continuada (CI/CD). Aquesta tecnologia és d'una gran utilitat, ja que gestiona els desplegaments als diversos entorns de programació presents a l'empresa, com ara CI, Test, Preproducció i Producció. A més de facilitar els desplegaments, Jenkins fa anàlisis exhaustives per assegurar-se que tot el codi es trobi en condicions correctes, identificant i resolent conflictes potencials. Amb aquest enfocament integral, Jenkins contribueix significativament a l'eficiència del cicle de vida del desenvolupament de *software* a l'ITeC, assegurant una integració i entrega fluides i sense problemes.

12.3. Disseny *backend*

Un cop vist com està organitzada i comunicada l'arquitectura de l'aplicació en aquest apartat es veurà en profunditat el disseny de la part del servidor. Es llistaran els mètodes públics dels controladors de l'API implicats en el projecte i també s'explicarà un diagrama de seqüència d'una de les funcionalitats per tal de veure com és el funcionament pràctic del servidor.

12.3.1. Mètodes dels controladors

En aquesta secció veure'm llistades les diferents crides que reben els controladors del *backend* per part de l'usuari que des del *frontend* les envia. Es tracta de peticions que usen operacions per agafar informació (GET), crear nova informació (POST), modificar informació existent (UPDATE) o eliminar (DELETE).

- Carregar línies de pressupost

URI	/obres/{idSisquillo}/arbre_nivells/nivells/{idNivell}/linies	
Query params		
Method	GET	
Body data		
Returns	Success Response	200 OK <pre>{ "id": 16998285, "amidament": 12.000000000000000, "idNivell": 13335799, "element": { "id": 17374570, "codiItec": "E61R5028", "nomcurt": null, "concepte": "PO", "teJustificacio": true, "teJustificacioPreus": true, "preu": 51.590000000000000, "unitatAmidament": "m"}, "ordre": 1, "hasChildren": true, "children": null, "type": "linia", "nom": "E61R5028 - Sellado fisuras mampostería mortero tricompcemento+epoxi", "teAmidamentsDetallats": false, "marca": null, "notes": null} }</pre>
	Error Response	404 Not Found <pre>{ "message": "No hi ha línies al nivell escollit" }</pre>

Taula 24: Definició d'operació - Carregar línies de Pressupost. Font: Elaboració pròpia.

Aquesta crida retorna les línies i la informació del nivell clicat per l'usuari a l'arbre de l'obra amb {idSisquillo} dins del mòdul de Pressupost.

- Carregar línies de seguiment

URI	/obres/{idSisquillo}/arbre_nivells/nivells/{idNivell}/linies	
Query params	{“periode”: “1”}	
Method	GET	
Body data		
Returns	Success Response	200 OK <pre>{ "id": 16998282, "idSisquillo": 12717621, "idNivell": 13335798, "element": { "id": 17374257, "codiItec": "E305E011", "nomcurt": "Hormigón simple cimentación, f'c=180 kg/cm2 consist. plástica, árido=< 10mm, vertido cubilote", "concepte": "PO", "teJustificacio": true, "teJustificacioPreus": true, "preu": 85.58000000000000, "unitatAmidament": "m3", "marca": null, "hasChildren": true, "children": null, "type": "linia", "idCertLinia": 12911521, "idCertPressupost": 8979004, "periode": 1, "periodeCreacioLinia": null, "amidPress": 12.000, "amidFinalEstimat": 12.000, "amidAdditiu": 12.000, "amidProporcional": 12.000, "importPress": 1026.9600000000000, "notes": "", "ultimPeriode": false, "amidDetallatOrigen": true, "teAmidPressupostat": false, "teAmidCertificat": false, "teAmidEstimat": false, "periodeLiniaEsUltimPeriode": false } }</pre>
	Error Response	404 Not Found <pre>{ "message": "No hi ha línies al nivell escollit" }</pre>

Taula 25: Definició d’operació - Carregar línies de Seguiment. Font: Elaboració pròpia.

En aquesta crida el funcionament és el mateix que l'anterior, però pren lloc en el mòdul de Seguiment, dins de l'URL hi trobem l'atribut intrínsec d'aquest mòdul que ens assenyala en quina etapa de la certificació es troba l'obra.

- **Marcar línia**

URI	/obres/{idSisquillo}/marcardesmarcarcolorlinies	
Query params		
Method	PUT	
Body data	{“colortriat”: “4e9262”}	
Returns	Success Response	200 OK {}
	Error Response	400 Bad Request {“message”: “Format de color no vàlid”}

Taula 26: Definició d'operació - Marcar línia. Font: Elaboració pròpia.

Aquesta crida s'encarrega de fer-li arribar al servidor el color triat per l'usuari i com s'utilitza també per la funcionalitat de desmarcar línia, la mateixa URL serveix també per indicar si cal marcar (atribut pren valor 'true') o desmarcar (pren valor 'false').

- **Comentar línia**

URI	/obres/{idSisquillo}/nivells/{idCapitol}/linies/{idLinia}/gestionoteslinia	
Query params		
Method	PUT	
Body data	{“notes”: “Comentari”}	
Returns	Success Response	200 OK {"id": 16998285, "amidament": 12.000000000000000, "idNivell": 13335799, "element": { " id": 17374570, "codiItec": "E61R5028", "nomcurt": null, "concepte": "PO", "teJustificacio": true, "teJustificacioPreus": true, "preu": 51.590000000000000, "unitatAmidament": "m"}, "ordre": 1, "hasChildren": true, "children": null, "type": "linia", "nom": "E61R5028 - Sellado fisuras mampostería mortero tricompcemento+epoxi", "teAmidamentsDetallats": false, "marca": null, "notes": "Comentari"}
	Error Response	404 Not Found {"message": “No existeix la línia amb el codi introduït”}

Taula 27: Definició d’operació – Marcar línia. Font: Elaboració pròpia.

En aquest cas d’ús, el funcionament del mètode del controlador és similar al de marcar o desmarcar línia, però no fa servir cap atribut sinó que simplement li fa arribar al controlador un formulari HTML amb la informació del comentari. El controlador aleshores actualitza la informació de la línia amb el valor del formulari.

- **Certificar pressupost**

URI	/obres/{idSisquillo}/crear_seguiment	
Query params		
Method	GET	
Body data		
Returns	Success Response	200 OK <pre>{ "editCrearSeguiment": { "id": 12720479, "numPeriode": 1, "adjudicatari": null, "dataCert": null, "dataInici": null, "dataTancament": null, "importAmidamentsDetallats": null, "entitatList": [{ "id": 1, "sigles": "B.O.E.", "nom": "Boletín Oficial del Estado" }, { "id": 32768, "sigles": "C.N.C.", "nom": "Confederación Nacional de la Construcción" }], "formulaList": [{ "id": 262144, "idFrpEntitat": 1, "idFrpTipusAdjudicacio": 1, "frpCoeficientsFormula": [{ "id": 196609, "idFrpTipusAdjudicacio": null, "valor": 0.3400000000000000, "nom": "P2", "valorPattern": "#,##0.00000" }], "numeroFormula": 14, "descripcio": "Dragados en terreno con predominio de roca", "termeFixe": 0.1500000000000000 }], "tipusAdjudicacioList": [{ "id": 1, "property": "frp.tipus.adjudicacio.public" }, { "id": 2, "property": "frp.tipus.adjudicacio.privat" }] } }</pre>
	Error Response	409 Conflict <pre>{ "message": "Ja existeix seguiment econòmic" }</pre>

Taula 28: Definició d'operació - Certificar Pressupost - GET. Font: Elaboració pròpia.

URI	/obres/{idSisquillo}/crear_seguiment	
Query params		
Method	POST	
Body data	{ "adjudicatari": "Marc", "datacert": 1706655600000, "datainici": 1704063600000, "dataTancament": 1706655600000 }	
Returns	Success Response	200 OK {"maxUploadSizeDocs": 10485760, "certPressupost": {"id": 8979005, "idSisquillo": 12718372, "idPressupost": 10817491, "idFrpFormula": null, "periode": 1, "datacert": 1706655600000, "datainici": 1704063600000, "dataTancament": 1706655600000, "certtancada": false, "certiniciada": false, "certaorigen": true, "adjudicatari": "Marc", "coefbaixa": 1.0000000000000000, "recalculpendent": null, "bloquejada": false, "potDesbloquejar": false, "tipusCertificacio": true}, "maxUploadSizeImages": 5242880, "needRecalculPartides": false, "ultimPeriode": true}
	Error Response	400 Bad Request {"message": "La data de certificació no pot ser anterior a la data de tancament"} 400 Bad Request {"message": "La data de tancament no pot ser anterior a la data d'inici"}

Taula 29: Definició d'operació – Certificar Pressupost – POST. Font: Elaboració pròpia.

El cas d'aquesta funcionalitat és una mica peculiar comparat amb la resta, el servidor rep primerament una crida GET encarregada de generar el model de dades amb els objectes necessaris per iniciar el seguiment del pressupost de l'obra amb l'identificador que rep a l'URL. Un cop generat aquest model de dades, rep una crida de tipus POST des de la qual es genera el primer període de certificació de l'obra.

- Cercar línia

URI	/obres/{idSisquillo}/elements/?page={}&perPage={}&search={}	
Query params	{“page”: “1”, “perPage”: “25”, “search”: “geotextil”}	
Method	GET	
Body data		
Returns	Success Response	200 OK {"resultatsCount": 1, "pages": 1, "perPage": 25, "capitols": [{"idCapitolPare": 13378119, "id": 13378123, "codi": "03", "descripcio": "CIMENTACIONES", "liniaCapitols": [{"id": 17374969, "ordre": null, "codi": "E7743281", "ua": "m2", "nom": "Memb.1 lám.HDPE perm.vapor n/resist.intemp.,masa=136-160g/m2,+refuerzo geotextil,sellad.cinta adhe", "preu": "12,75"}]]}
	Error Response	404 Not Found {“message”: “No s’ha trobat cap línia”}

Taula 30: Definició d'operació - Cercar línia. Font: Elaboració pròpia.

Aquesta crida ve determinada pels atributs que l'usuari pot configurar en generar la cerca, si clica a un número de pàgina específic el servidor ho rep i retorna les línies necessàries. Aquesta quantitat de línies ve determinada pel valor que prengui l'atribut *perPage*. I l'últim camp, és el que li transmet al controlador la combinació de caràcters que ha d'aparèixer en les línies que retorni.

- **Crear nivell**

URI	/obres/{idSisquillo}/nivells	
Query params		
Method	POST	
Body data	{“id”: “GG”, “descripcio”: “Nivell nou”}	
Returns	Success Response	200 OK {"id": 13386517, "idSisquillo": null, "descripcio": "Nivell nou", "import": 0E-14, "tnoId": "GG", "camiOrdinal": "01.GG", "idNivellPare": 13378118, "hasChildren": false, "hasLinies": false, "typeChildren": "", "children": null, "type": "nivell", "nom": "GG - Nivell nou", "ultimNivellArbre": false, "nivellMarcadors": null, "idvista": 0}
	Error Response	400 Bad Request {“message”: “Ja existeix un nivell amb el codi introduït”}

Taula 31: Definició d'operació - Crear Nivell. Font: Elaboració pròpia.

En aquesta crida el sistema rep el formulari que ha omplert el client i genera una nova entitat Nivell dins de l'obra que s'està pressuposant des de TCQI. A continuació el sistema organitza l'arbre de l'obra per mostrar el nivell creat i que els ja existents quedin organitzats correctament.

12.3.2. Esquema de capes del *backend*

Havent analitzat les crides públiques que les funcionalitats optimitzades en el projecte utilitzen, cal entendre quin procés segueixen per tal de modificar, afegir o recollir les dades. Per aquest motiu en aquest apartat es veurà superficialment un diagrama amb els arxius implicats per les diferents funcions, arxius de les capes explicades en l'apartat 12.1.1. Començant pel controlador que rep el mètode públic, després es contacta amb la capa de negoci pertinent, quan ja s'ha efectuat la lògica necessària de l'operació s'accedeix als repositoris blaze per accedir a la base de dades recollint la informació especificada en les vistes blaze.

A la figura a continuació es mostra les classes implicades pels mètodes llistats a l'apartat anterior. S'ha separat en dues pàgines per tal de poder llegir cada classe i funció correctament, però tot forma part del mateix esquema. Diagrama que mostrar una vista esquemàtica i sense dades per poder tenir una visió global del sistema. Tampoc s'han afegit les diferents vistes blaze, ja que es creen varies per cada funcionalitat i s'ha volgut mantenir l'esquema el més llegible possible. Els controladors els podem veure mostrats de color roig i contenen les funcions dels mètodes públics. En color taronja trobem els arxius de negoci que contenen la lògica operacional. Finalment, mostrats de color groc trobem els diferents repositoris blaze encarregats de la comunicació amb les diferents taules de la base de dades

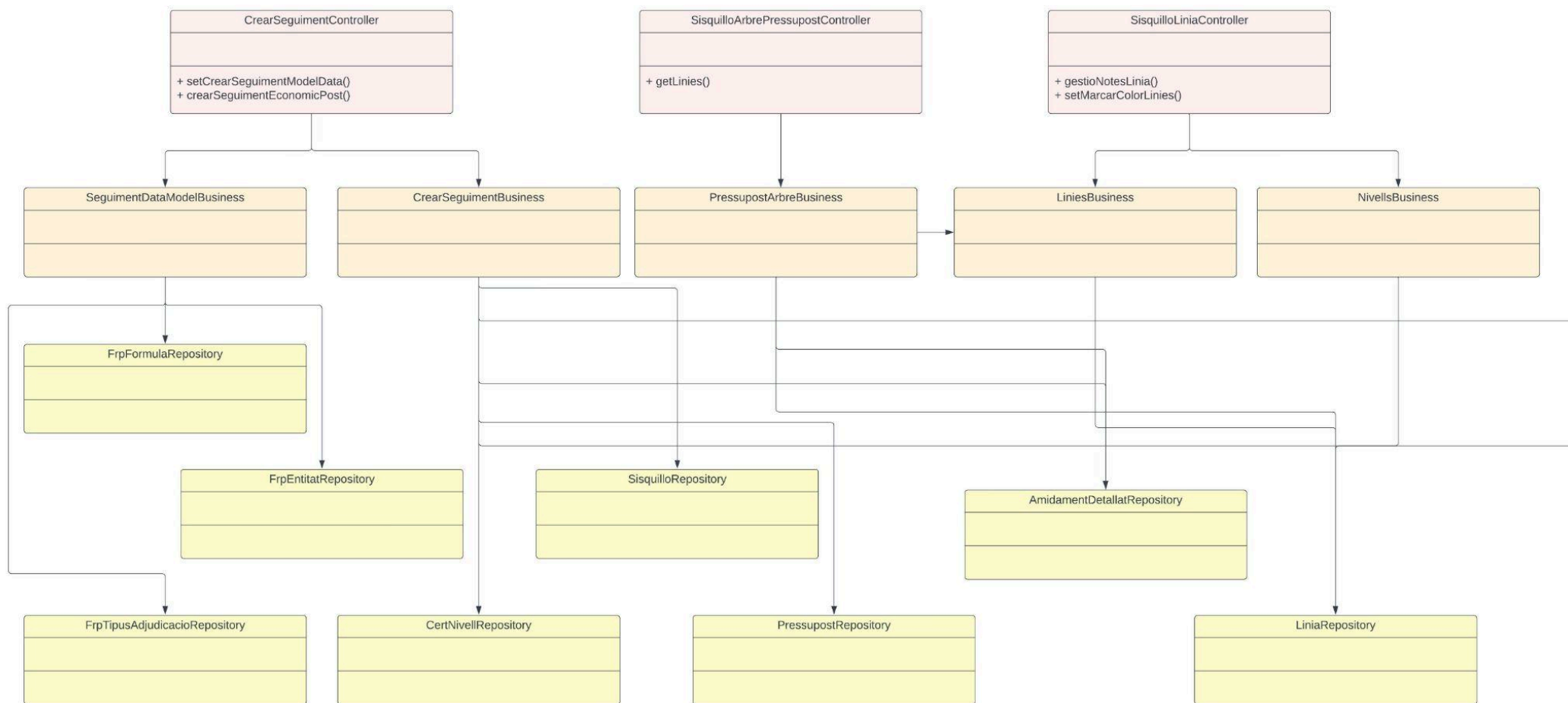


Figura 13: Esquema capes del *backend* per les funcions del projecte - 1a part. Font: Elaboració pròpia.

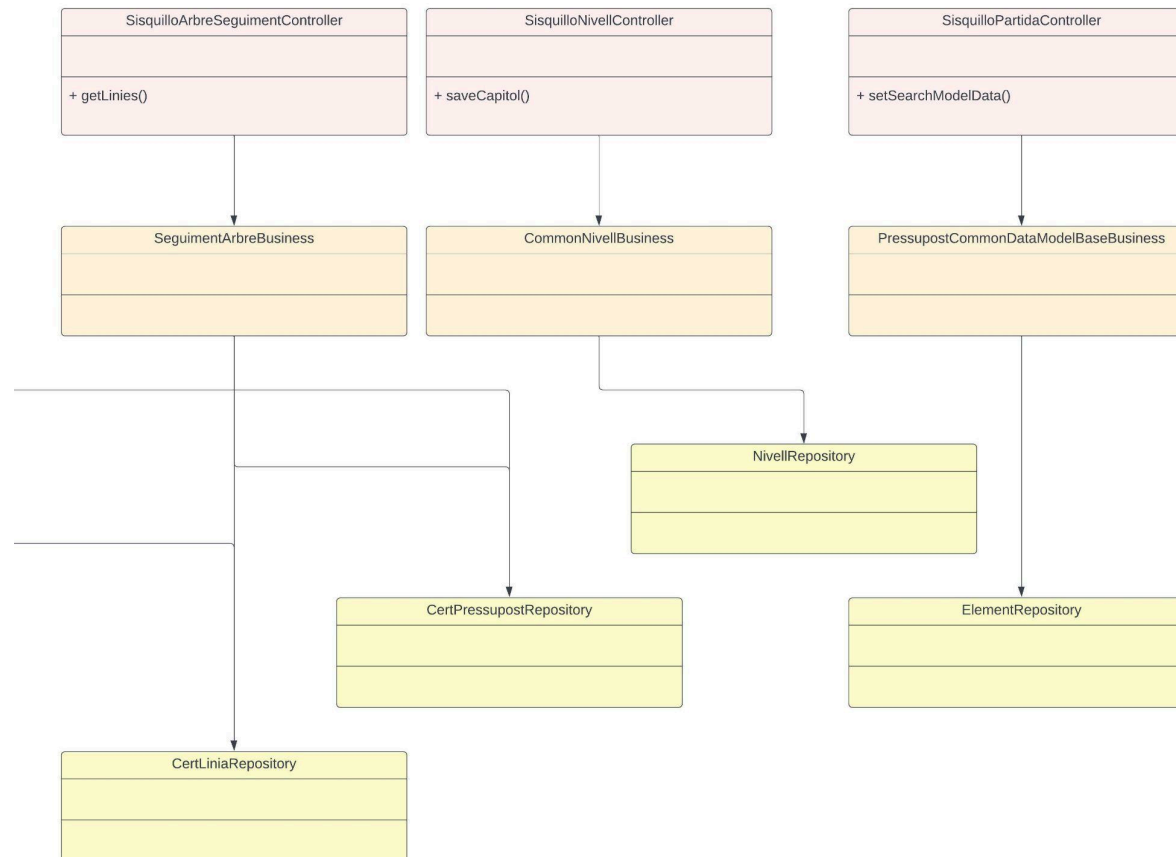


Figura 14: Esquema capes del *backend* per les funcions del projecte - 2a part. Font: Elaboració pròpia.

12.3.3. Diagrama de classes

Havent vist de forma genèrica com està estructurada la part de servidor de l'aplicació, s'ha volgut profunditzar en el funcionament i la comunicació entre les capes a través d'analitzar de la funcionalitat de carregar línies en el mòdul de Pressupost. Per tal d'entendre com funciona, en aquesta secció utilitzarem un diagrama del domini per veure amb precisió quines classes i funcions utilitza. En aquest diagrama es poden veure també les vistes blaze específiques que en l'apartat anterior no s'han pogut veure. Voldria destacar que a la figura que es veu a continuació apareixen únicament aquelles funcions i atributs necessàries per a la funció analitzada.

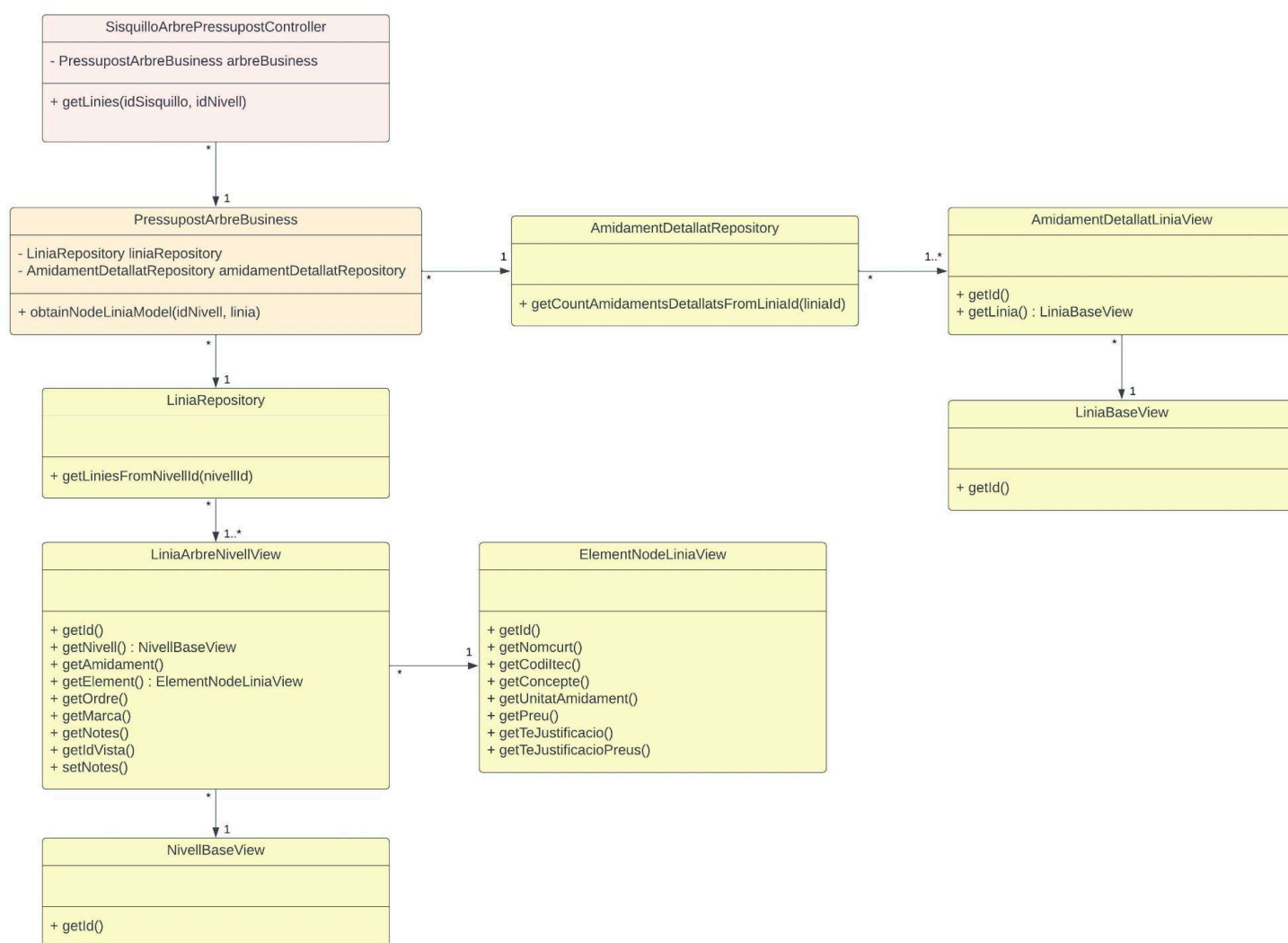


Figura 15: Diagrama del domini – Carregar línies de pressupost. Font: Elaboració pròpia.

12.3.4. Diagrames de seqüència

Seguint l'exemple descrit en l'apartat anterior, en aquesta secció es mostra els diagrames de seqüència que la funcionalitat segueix. L'operació s'inicia un cop el servidor rep la crida de tipus GET que l'usuari fa a través de l'URL /obres/{idSisquillo}/arbre_nivells/nivells/{idNivell}/linies. Aquest mètode necessita agafar totes les línies que conté el nivell clicat per llistar-les al *frontend*. A continuació es veu la figura amb la representació del funcionament de l'operació.

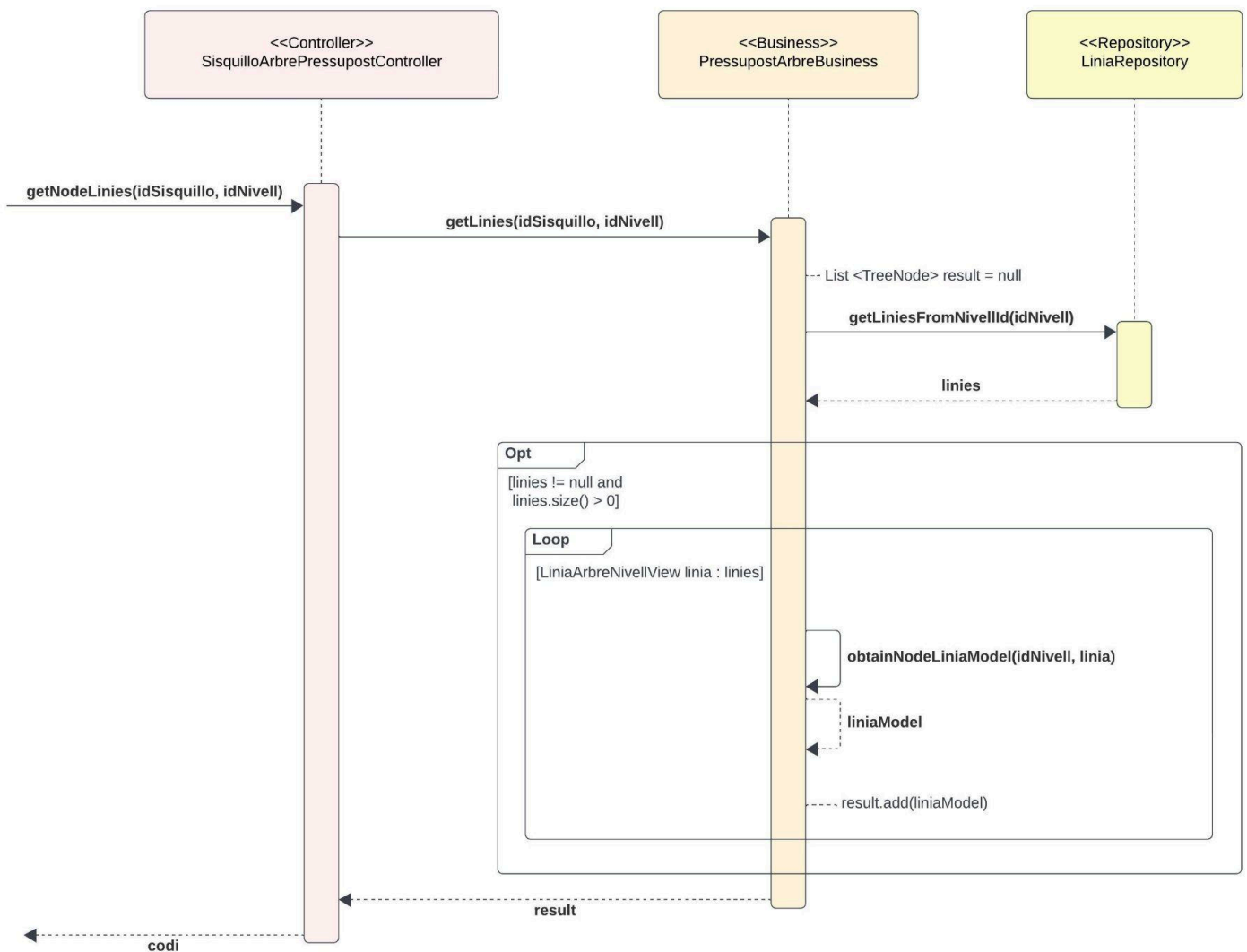


Figura 16: Diagrama de seqüència - Carregar línies de pressupost. Font: Elaboració pròpia.

La funció navega des del controlador fins al business de l'arbre de Pressupost, ho fa per poder tornar amb la informació de totes les línies del nivell amb l'identificador que rep per paràmetre. Per fer-ho, primer declara una llista de nodes que acabarà sent el que retorni la funció. Després accedeix al repositori Blaze on s'agafen totes les línies del nivell i comprova que ha obtingut resultats de l'accés. Si és així, itera

per totes les línies (en format *LiniaArbreNivellView*, vista de blaze) de la llista i crida a la funció d'obtenir el model. Per cadascuna de les línies van afegint el seu model a la llista result. Aquest conjunt de models de dades és el que acabarà retornant cap al controlador i aquest cap al *frontend*.

Com es pot veure a la figura anterior, hi ha una funció dins l'execució al business que es crida al mateix business. Es tracta de la càrrega dels diferents atributs de la línia al model. És un procés que necessita fer l'operació, ja que el *frontend* està configurat per rebre el model de dades sencer i dins del *scope* del projecte no es podia canviar res que no fos *backend*. A continuació veiem el diagrama de l'operació.

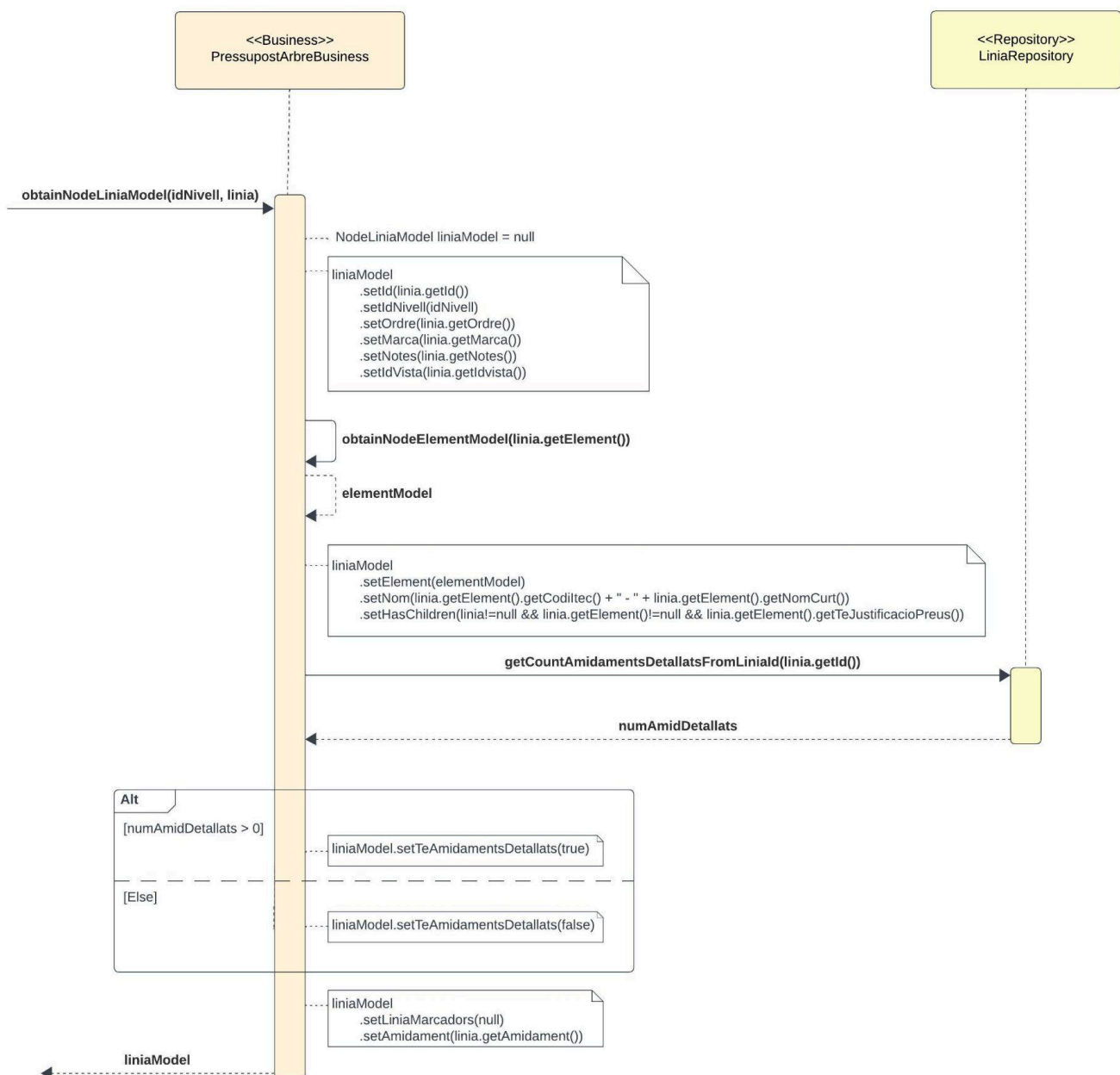


Figura 17: Diagrama de seqüència - Obtenir model de la línia. Font: Elaboració pròpia.

La funció comença passant cada atribut de la línia cap al model. Abans de poder afegir un dels atributs importants del model de la línia, l'atribut que determina si cal assenyalar que l'amidament de l'element està justificat a través d'un amidament detallat, el business ha de comprovar si aquella línia té assignada alguna instància d'amidament detallat. Per això procedeix al repositori blaze pertinent i recupera el compte de totes les instàncies d'amidament detallat que tingui la línia. Quan retorna la funció al business, comprova si el número és positiu i marcà el booleà com toqui.

A més, de manera similar a l'anterior, aquesta funció es troba que en un moment ha d'accedir a la informació de l'element. Aquell que conté la línia per generar el model i que el *frontend* pugui llegir totes les dades que conté. Per això torna a cridar a una funció del mateix business on es farà aquest mapeig de l'element al model tal com es pot veure en el diagrama a continuació.

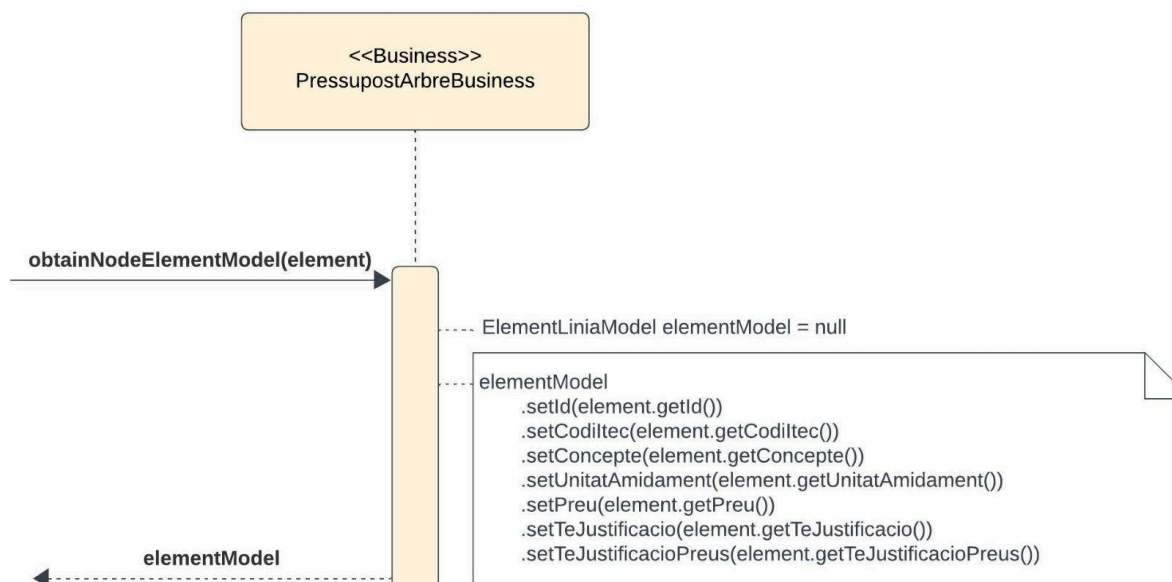


Figura 18: Diagrama de seqüència - Obtenir model de l'element. Font: Elaboració pròpia.

12.4. Esquema Base de Dades

Acabant la secció de Disseny, es mostra com està definit l'esquema de la base de dades. Es podrà llegir a continuació les diferents taules utilitzades dins el marc del projecte. Cal tenir en compte que el sistema de bases de dades usat per l'empresa és més extens que la representació d'aquest apartat, però com a la resta de memòria del projecte únicament ens centrarem en les taules significatives per aquest. Per cada taula llistada s'afegeixen els atributs que la formen, on apareixen atributs subratllats indicant que és la clau primària. A més es pot veure les especificacions d'algunes restriccions o de claus foranes, que defineixen relacions entre taules del sistema.

Moneda(id, descripcio, simbol, codiiso, genere, fracciogenera)

Sisquillo(id, descripcio, codi, data_creacio, usr_creacio, data_modificacio, usr_modificacio, id_moneda, ds, dc, notes)

CHECK {id_moneda} is NOT NULL

FOREIGN KEY {id_moneda} REFERENCES moneda(id)

Pressupost(id, id_sisquillo, num_max_nivells, per_iva, per_despeses_generals, per_benefici_industrial)

CHECK {id_sisquillo} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

Nivell(id, id_sisquillo, id_pare, descripcio, tno_id, import, cami_ordinal)

CHECK {id_sisquillo} is NOT NULL

CHECK {tno_id} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_pare} REFERENCES nivell(id)

Element(id, id_sisquillo, codi_itec, nomcurt, unitat_amidament, preu, total_madobra, total_maquinaria, total_material, total_sense_despeses, concepte, banc_origen)

CHECK {id_sisquillo} is NOT NULL

CHECK {codi_itec} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

Linia(id, id_sisquillo, id_nivell, amidament, notes, marca, id_element, ordre)

CHECK {id_sisquillo} is NOT NULL

CHECK {id_nivell} is NOT NULL

CHECK {id_element} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_nivell} REFERENCES nivell(id)

FOREIGN KEY {id_element} REFERENCES element(id)

FrpEntitat(id, sigles, nom)

CHECK {sigles} is NOT NULL

CHECK {nom} is NOT NULL

FrpTipusAdjudicacio(id, property)

CHECK {property} is NOT NULL

FrpIndex(id, id_entitat, nom, descripcio)

CHECK {id_entitat} is NOT NULL

CHECK {nom} is NOT NULL

FOREIGN KEY {id_entitat} REFERENCES frpEntitat(id)

FrpFormula(id, id_entitat, id_tipus_adjudicacio, numero_formula, descripcio, terme_fixe)

CHECK {id_entitat} is NOT NULL

CHECK {id_tipus_adjudicacio} is NOT NULL

CHECK {numero_formula} is NOT NULL

CHECK {descripcio} is NOT NULL

CHECK {terme_fixe} is NOT NULL

FOREIGN KEY {id_entitat} REFERENCES frpEntitat(id)

FOREIGN KEY {id_tipus_adjudicacio} REFERENCES frpTipusAdjudicacio(id)

FrpCoeficientFormula(id, id_formula, id_index, id_tipus_adjudicacio, valor, nom)

CHECK {id_formula} is NOT NULL

CHECK {id_index} is NOT NULL

CHECK {id_tipus_adjudicacio} is NOT NULL

CHECK {valor} is NOT NULL

CHECK {nom} is NOT NULL

FOREIGN KEY {id_formula} REFERENCES formula(id)

FOREIGN KEY {id_index} REFERENCES frpIndex(id)

FOREIGN KEY {id_tipus_adjudicacio} REFERENCES frpTipusAdjudicacio(id)

CertPressupost(id, id_pressupost, id_sisquillo, periode, datacert, datainici, datafinal, adjudicatari, id_formula, data_adjudicacio)

CHECK {id_pressupost} is NOT NULL

CHECK {id_sisquillo} is NOT NULL

CHECK {periode} is NOT NULL

CHECK {datacert} is NOT NULL

CHECK {datainici} is NOT NULL

CHECK {datafinal} is NOT NULL

FOREIGN KEY {id_pressupost} REFERENCES pressupost(id)

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_formula} REFERENCES frpFormula(id)

CertNivell(id, id_sisquillo, id_certpressupost, id_nivell, periode, certificacio, teoric, proporcional, estimacio)

CHECK {id_sisquillo} is NOT NULL

CHECK {id_certpressupost} is NOT NULL

CHECK {id_nivell} is NOT NULL

CHECK {periode} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_certpressupost} REFERENCES certPressupost(id)

FOREIGN KEY {id_nivell} REFERENCES nivell(id)

CertLinia(id, id_sisquillo, id_certnivell, id_linia, periode, certificacio, teoric, proporcional, estimacio, amiddetallorigen)

CHECK {id_sisquillo} is NOT NULL

CHECK {id_certnivell} is NOT NULL

CHECK {id_linia} is NOT NULL

CHECK {periode} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_certnivell} REFERENCES certNivell(id)

FOREIGN KEY {id_linia} REFERENCES linia(id)

CertElement(id, id_sisquillo, id_element, descripcio, periode, periode_aplec, amidament_aplec_origen, id_certpressupost)

CHECK {id_sisquillo} is NOT NULL

CHECK {id_element} is NOT NULL

CHECK {periode} is NOT NULL

CHECK {periode_aplec} is NOT NULL

CHECK {id_certpressupost} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_element} REFERENCES element(id)

FOREIGN KEY {id_certpressupost} REFERENCES certPressupost(id)

AmidamentDetallat(id, id_sisquillo, id_linia, id_certlinia, contingut, total, color, formula, num_fila)

CHECK {id_sisquillo} is NOT NULL

CHECK {num_fila} is NOT NULL

FOREIGN KEY {id_sisquillo} REFERENCES sisquillo(id)

FOREIGN KEY {id_linia} REFERENCES linia(id)

FOREIGN KEY {id_certlinia} REFERENCES certlinia(id)

13. Implementació

Un cop acabades les especificacions del sistema i el disseny de les modificacions a fer, ara ens focalitzarem en la implementació dels canvis, les tecnologies i llenguatges utilitzats i detalls tinguts en compte durant el desenvolupament. Per tal d'entendre millor la secció, anirem passant per les diferents tasques que ha tingut el projecte i valorant com ha sigut la implementació de cadascuna

13.1. Entorns de desenvolupament

Com prèviament s'ha explicat, el treball es fa a ITeC, a la seva aplicació TCQi. Es tracta d'un sistema extens i modular que consta de diferents repositoris per cadascun d'aquests mòduls i les seves funcionalitats. Tot el codi que ha calgut desenvolupar havia de ser integrat orgànicament dins l'ecosistema de TCQi existent. Per assegurar que els nous canvis no fessin deixar de funcionar correctament l'aplicació, a cadascun dels repositoris Git on calia intervenir es va generar una branca anomenada *feature/base2*. Des d'aquesta subbranca de *develop* cada desenvolupador generava una branca amb el codi identificador de la tasca redmine pertinent i des d'allà s'anava modificant amb compte les diferents crides i accessos.

Totes les implementacions es fan a un primer entorn denominat **CI (Continuous Integration)**. Aquest entorn és on es desenvolupen totes les funcionalitats i mitjançant les branques es preparen els canvis per acabar fent *merge* a *develop*. Un cop la branca *develop* està actualitzada la tecnologia automatitzada de Jenkins passa el codi a l'entorn de **Test**, on es proven per primer cop les funcionalitats implementades i es revisen si cal amb nous redmines. Aquest procés de pujada el fa cada dia a les 4:00 h, per tots els repositoris de l'empresa.

Després ens trobem l'entorn de **Preproducció**, aquí trobem les versions dels productes un cop han estat revisades pels desenvolupadors i els testers. En aquest estat es realitzen comprovacions amb experts de la construcció i usant una còpia de la base de dades de l'entorn final. És on s'acostuma a contactar el client i fer les primeres mostres reals del producte que ha contractat.

Finalment, trobem l'entorn de **Producció** on l'aplicació està completa i funciona correctament amb totes les noves implementacions incorporades. El codi surt de l'entorn de test, quan ha passat l'anàlisi a l'entorn de preproducció. Trobem el codi final i estable, preparat per ser utilitzat per l'usuari. Com prèviament s'ha explicat, tots els productes que ofereix ITeC no van junts amb les mateixes llicències, depenent del contracte que tingui l'usuari amb l'empresa podrà accedir a uns o altres mòduls.

A banda dels entorns de desenvolupament, també cal entendre la manera que té ITeC per gestionar les bases de dades. Per cada desenvolupador es genera una còpia local de la base de dades de l'entorn de CI amb tots els permisos, d'aquesta forma cadascú té llibertat màxima per comprovar, provar i implementar el que necessiti per cada tasca que tingui assignada sense afectar a la feina de la resta de companys.

Aquest projecte, en tractar d'optimitzar funcions ja existents, s'ha desenvolupat tot dins l'entorn de CI. En el futur, quan totes les funcionalitats dels mòduls de pressupost i certificació estiguin completes, es donaria la *feature* com a realitzada i s'iniciaria el procés d'incorporar les modificacions cap a Producció. Malgrat això, com es tracta d'un projecte intern, sense clients i pensat per millorar el rendiment dels productes existents no s'ha seguit encara el procés d'enviar el codi pels diferents entorns.

13.2. Primer *sprint*

En el primer *sprint* es volien implementar les millores de les funcionalitats relacionades amb les línies del mòdul de pressupost. Tanmateix, no es constava amb la difícil corba d'aprenentatge de la tecnologia Blaze i de la incorporació d'aquesta dins el sistema de TCQi. Certament, dins l'empresa no hi havia ningú que conegués ni hagués utilitzat en profunditat l'eina i ens va tocar anar descobrint una mica seguint prova i error com calia declarar cada classe, com incorporar els repositoris amb les vistes, com funcionava el llenguatge DSL, entre altres inconvenients que vam trobar. Per aquest motiu dins del primer *sprint* únicament es va desenvolupar la funció L1: Càrrega de les línies de Pressupost.

[L1] Càrrega de les línies d'un nivell al mòdul de Pressupost

Aquesta tasca ha necessitat crear diferents vistes blaze, retallant la quantitat d'atributs per taula recollits pel *backend*. S'ha necessitat generar dos arxius de repositori blaze amb funcions d'accés a la BD. Pel que fa a la lògica de l'operació, s'ha hagut d'adaptar les funcions existents de mapeig entre la informació emmagatzemada dins els esquemes de la base de dades i els models d'informació preparats per què el *frontend* de l'aplicació mostres la informació correctament. Va costar entendre el funcionament de l'operació perquè tenia parts repartides entre els repositoris git de l'empresa i diferents arxius i es tractava de la primera implementació que realitzava a TCQi. Juntament amb el desconeixement de Blaze, ha resultat en aquesta gran desviació horària.

13.3. Segon *sprint*

En el segon *sprint* es van iniciar unes petites tasques que van venir derivades del primer, no estaven dins la planificació inicial, però es va veure que compartien molt sentit per l'usuari amb la tasca L1 i es va decidir deixar-les implementades en bloc abans de continuar amb altres aspectes de l'aplicació. A més d'aquestes dues tasques, com cada funcionalitat està organitzada diferent i els càlculs interns són un món, es va decidir que m'encarregués de la funcionalitat de carregar línies en el mòdul de seguiment perquè els càlculs no, però el comportament era similar a la tasca L1.

[L5] Marcar color de fons en una o més línies del mòdul de Pressupost

Aquesta funcionalitat es va veure que era necessària, ja que anava conjuntament amb la línia. Es va veure que un atribut que la línia carregava era la marca del color i es va necessitar que la funcionalitat estigués operativa per poder eliminar codi duplicat que utilitzaven les dues funcionalitats. El cas particular d'aquesta funció era gestionar la quantitat de línies que l'usuari volia marcar i la connexió amb quan ho demanava, juntament amb el color escollit i guardar-ho de forma específica perquè el *frontend* ho pintés correctament.

[L6] Afegir un comentari en una línia del mòdul de Pressupost

La tasca L3 va venir de similar forma que l'anterior. Quan un dels responsables del projecte estava comprovant els resultats de l'optimització de L1 va veure que la funció que gestionava les notes calia treballar-la de la mateixa forma i se li va donar prioritat.

[S1] Càrrega de les línies certificades d'un nivell al mòdul de Certificació

Com al primer *sprint* em vaig encarregar de la càrrega de les línies del mòdul de pressupost, em van assignar aquesta tasca. Principalment, va caldre afegir repositoris blaze semblants als creats per L1, però en la seva versió certificada pel mòdul de Seguiment. D'igual forma que la versió corresponent a l'altre mòdul, la funcionalitat comença recuperant la informació per passar-la posteriorment en el model de dades pertinent per mostrar al *frontend*.

13.4. Tercer *sprint*

El tercer *sprint* estava preparat per unes tasques diferents, però per una complexitat tècnica en les funcionalitats de copiar i enganxar, es va modificar la planificació. Aquestes funcions constaven de crides recursives a la base de dades i de difícil conversió al llenguatge que Blaze ofereix. Ens vam reunir amb els companys de l'empresa externa que ens han fet l'acompanyament amb el projecte i es va decidir apartar temporalment aquestes tasques i que els desenvolupadors ens centréssim en altres.

[CL1] Cerca de línies dins l'obra

Aquesta tasca se'm va assignar per la relació aparent que conté amb les línies, no obstant els càlculs de cerca estan configurats a través de la coincidència amb els textos que conté l'element de la línia. Es van generar nous arxius específics per la funcionalitat i les funcions d'accés amb les comparacions de text.

[N1] Crear un nou nivell dins l'obra

Aquesta funcionalitat constava primer d'afegir el codi i la descripció del nivell dins d'una nova instància i posteriorment recórrer l'arbre reordenant els camins ordinals dels nodes contigus i reorganitzant els ordres de l'arbre.

13.5. Quart *sprint*

Per aquest *sprint* final des de l'empresa es va decidir continuar amb una altra línia que no era la planificada, però que encaixava més amb el progrés que els hi interessava pel projecte. Inicialment, la idea era comprendre primer les millores del mòdul de Pressupost i posteriorment del de Certificació, però es va considerar que seria millor paral·lelitzar, ja que com el cas de la càrrega de línies, algunes funcionalitats tenien similituds. Per aquest motiu se'm va assignar la tasca de la funcionalitat de Certificar Pressupost, i ha estat l'única realitzada per la seva complexitat. Ha suposat la finalització del desenvolupament de codi del projecte i ha resultat una desviació de 7 hores respecte a la planificació inicial.

[S2] Creació del seguiment d'una obra pressupostada.

L'última funcionalitat ha estat la més diferent de la resta, ja que tracta del traspàs d'informació d'una obra acabada de pressupostar cap al mòdul de Certificació per tal d'iniciar el seguiment. Està separada en dues crides, una primera que s'encarrega de recollir informació de l'obra de pressupost i emmagatzemar les dades que l'usuari introdueix en el formulari de creació de seguiment. La segona crida va recorrent tota la informació obtinguda del mòdul de Pressupost i va creant les contraparts necessàries en el mòdul de Seguiment, generant tots els arxius Blaze corresponents i accedint a la base de dades introduint els elements, línies i capítols en bloc.

14. Validacions

Després de portar a terme la implementació del nostre projecte, ara ens proposem analitzar amb detall si hem assolit amb èxit els objectius que havíem establert i els requisits inicials. Hem dut a terme una revisió exhaustiva tant dels requisits funcionals com dels no funcionals. A continuació, procedim a examinar de manera individual els resultats aconseguits en els diferents escenaris dels requisits, acompanyant-ho amb una detallada descripció de les observacions rellevants.

La importància d'aquesta fase radica en el fet que els requisits juguen un paper fonamental per garantir la utilitat del sistema. És, per tant, d'una importància crítica assegurar-nos que aquests requisits s'han satisfet.

Per fer aquesta validació, revisarem el procediment utilitzat per validar els requisits i posteriorment analitzarem en detall tots ells, siguin principals o secundaris, amb l'objectiu de determinar si s'han complert adequadament. En cas contrari, durem a terme una investigació per entendre si aquestes mancances es deuen a una definició insuficient dels objectius inicials o si han experimentat canvis al llarg del desenvolupament del projecte.

14.1. Procediment de les validacions

Dins de l'empresa, s'apliquen dos nivells de validació de requisits per a tot el *software* desenvolupat. El primer nivell es fonamenta en la *pipeline* d'integració contínua i distribució contínua. Aquest enfocament implica la creació de proves automatitzades essencials en diversos punts de l'aplicació. Aquestes proves s'executen de forma automàtica cada vegada que es requereixi realitzar una distribució de l'aplicació en els diferents entorns abans esmentats.

Després de concloure el desenvolupament d'una funcionalitat, es du a terme la segona fase de validació, en la qual els experts en construcció de l'ITeC procedeixen a verificar exhaustivament el correcte funcionament de l'aplicació en l'entorn de proves. En cas que aquesta validació obtingui resultats satisfactoris, es consideren finalitzades les tasques associades amb èxit. No obstant això, si la validació no compleix amb les expectatives, la tasca afectada es reincorpora al *backlog* o a l'*sprint* actual, en funció de la seva urgència, amb l'objectiu de revisar i ajustar la seva implementació per assegurar-ne el compliment dels requisits especificats.

Aquest procés permet mantenir un alt estàndard de qualitat i eficàcia en el desenvolupament de l'aplicació. Des de la perspectiva d'ITeC, es considera que aquests dos nivells de validació ofereixen una base adequada per determinar la correcció del comportament de les funcionalitats desenvolupades, prenent en consideració tots els casos possibles. Aquest enfocament reforça la confiança en la robustesa i la integritat del *software*, assegurant que les diverses funcionalitats compleixin amb els requisits establerts i ofereixin una experiència d'usuari consistent i fiable.

En relació amb el mateix projecte, estava previst realitzar una sèrie de proves mitjançant l'eina Gatling; no obstant això, la integració d'aquests tests va ser posposada per decisió de l'empresa. A conseqüència de la no implementació d'aquestes proves, es va optar per una estratègia alternativa amb l'objectiu de verificar l'eficàcia de les millores implementades i, per tant, avaluar els resultats dels requisits pertinents. Així doncs, es va introduir una mesura del temps abans i després de les execucions de les funcions, fent diverses iteracions per a cada funció i considerant conjunts de dades significatius.

En cas que les mesures de temps obtingudes resultessin satisfactòries, es considerava que la tasca es donava per finalitzada i es procedia amb la següent fase del projecte. Aquesta metodologia proporciona una avaluació pràctica del rendiment sense recórrer als tests planificats inicialment, assegurant alhora la comprovació del compliment dels requisits específics.

14.2. Validacions dels requisits principals

En aquest apartat es llistaran els diferents requisits principals i els diversos escenaris on es pot mesurar la satisfacció i la resolució de cadascun dels requisits. A les taules s'inclou l'escenari en si, una petita observació i el resultat de la prova.

1. Permetre a l'usuari accedir a les dades de les seves obres més ràpidament		
Escenari	Resultat	Observacions
S'executa diverses vegades la funcionalitat 'carregar línies de pressupost' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.
S'executa diverses vegades la funcionalitat 'carregar línies de seguiment' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.

S'executa diverses vegades la funcionalitat 'marcar línia' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.
S'executa diverses vegades la funcionalitat 'comentar línia' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.
S'executa diverses vegades la funcionalitat 'certificar pressupost' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.
S'executa diverses vegades la funcionalitat 'cercar línia' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.
S'executa diverses vegades la funcionalitat 'crear nivell' un cop aplicades les millores.	Favorable	El temps de resposta obtingut amb les comprovacions queda per sota el segon.

Taula 32: Validació requisit principal 1. Font: Elaboració pròpia.

2. El canvi de tecnologia utilitzada en els accessos no ha d'afectar el funcionament de l'aplicació		
Escenari	Resultat	Observacions
S'escull una obra carregada amb informació i s'executa cada funcionalitat optimitzada en la versió del programa amb millores i sense.	Favorable	Cadascuna de les comparacions de resultats coincideix i els punts observats durant l'execució contenen la mateixa informació.

Taula 33: Validació requisit principal 2. Font: Elaboració pròpia.

14.3. Validacions dels requisits secundaris

En aquesta secció s'estudia els diferents requisits no funcionals addicionals, definits a l'inici de la memòria i com han acabat resultant i complint-se al llarg del treball. Per fer aquest estudi s'ha emprat la taula d'escenari, resultats i observacions d'igual forma que en l'apartat anterior.

1. Requisits de personalització i internalització		
Escenari	Resultat	Observacions
Els textos de l'aplicació estan definits per tres idiomes: català, castellà i anglès.	No aplicable	En cap de les tasques executades s'ha acabat modificant text pla de l'aplicació que calgués aplicar les configuracions d'idiomes.

Taula 34: Validació requisit secundari 1. Font: Elaboració pròpia.

2. Requisits de velocitat i latència		
Escenari	Resultat	Observacions
Els temps d'execució de les funcionalitats optimitzades són inferiors a un segon.	Favorable	Totes les optimitzacions realitzades a les tasques han resultat satisfactòries i per sota del segon. El temps varia depenent de la quantitat d'informació que contingui l'obra.

Taula 35: Validació requisit secundari 2. Font: Elaboració pròpia.

3. Requisits de precisió i exactitud		
Escenari	Resultat	Observacions
Els nombres es mostraran sempre amb una exactitud de dos decimals, o el que la configuració de l'usuari assenyali.	Favorable	No s'han obviat els valors que marquen l'exactitud dels càlculs en la informació a recollir de les bases de dades i s'han mantingut la precisió en cada mostra de valors numèrics.

Taula 36: Validació requisit secundari 3. Font: Elaboració pròpia.

4. Requisits de confiança i disponibilitat		
Escenari	Resultat	Observacions
Les incidències que ha d'haver-hi en un any han de ser inferiors a cinc. A part la disponibilitat del sistema ha de ser del 95% del temps total.	No aplicable	No es pot analitzar aquest escenari durant el desenvolupament del projecte. Ha estat definit com a requisit no funcional per la importància del criteri dins l'empresa.

Taula 37: Validació requisit secundari 4. Font: Elaboració pròpia.

5. Requisits de longevitat		
Escenari	Resultat	Observacions
S'espera que el producte tingui una vida útil de mínim quinze anys.	No aplicable	No es pot analitzar aquest escenari durant el desenvolupament del projecte. Ha estat definit com a requisit no funcional per la importància del criteri dins l'empresa.

Taula 38: Validació requisit secundari 5. Font: Elaboració pròpia.

6. Requisits de manteniment		
Escenari	Resultat	Observacions
En cas de detectar un error, cal solucionar i pujar a producció en menys de 92 hores.	No aplicable	El desenvolupament de les millores no ha estat afegit en cap moment a l'entorn de producció i, per tant, no s'ha aplicat aquest criteri de manteniment.

Taula 39: Validació requisit secundari 6. Font: Elaboració pròpia.